LAMP: Linear Verification of Matrix Multiplication via Proximity Testing

Anonymous Submission

***Abstract*—Verifiable computation systems often need to prove large matrix multiplication statements, but a direct SNARK arithmetization of a $k \times k$ product requires $\mathcal{O}(k^3)$ constraints. Freivalds' randomized check reduces the algebraic computation to vector-matrix products, but proving those products inside a SNARK still costs $\mathcal{O}(k^2)$ constraints.**

We present LAMP, a matrix-multiplication checking protocol that combines Freivalds' randomized check with proximity testing over linear error-correcting codes. The prover commits to encoded matrices and intermediate vectors before the sampled query positions are derived. The CP-SNARK circuit then checks only the sampled codeword positions and commits to the values used inside the circuit, while Merkle openings and CP-Link proofs ensure consistency between the in-circuit witnesses and the externally committed values. We prove soundness for this committed-input setting under the soundness of the SNARK backend, the binding of the commitments, the correctness of the CP-Link checks, and the relative distance and unique-decoding radius of the Reed–Solomon code.

For a fixed number t of sampled positions, the main in-circuit SNARK relation has $\mathcal{O}(tk + E_{\mathsf{code}})$ constraints, with additional $\mathcal{O}(t \log n) + E_{\mathsf{link}}(k, t)$ backend work for Merkle openings and CP-Link checks. We implement LAMP in Go and compare it with a Freivalds-based SNARK circuit. In the matrix benchmark at $k = 2^{12}$, LAMP reduces the constraint count by $9.68\times$ and shortens proof generation time by $5.88\times$; verification remains approximately $0.13\,\mathrm{s}$ across the measured range.

***Index Terms*—Verifiable computation, SNARKs, Proximity testing, Linear codes, Freivalds' algorithm, Merkle trees, Commitment linking**

1. Introduction

Matrix multiplication is a core computational primitive underlying many large-scale computations, including numerical linear algebra, signal processing, computer graphics, and artificial intelligence. Consequently, many applications require verifiability for matrix products while keeping the underlying matrices private. Zero-knowledge proofs provide a cryptographic mechanism for this goal: a prover can convince a verifier that a statement is correct without revealing any secret information that witnesses the statement.

Applying general-purpose ZKP systems directly to large matrix multiplication, however, remains expensive. Succinct proof systems such as [1]–[4] prove relations after the computation has been represented as an arithmetic circuit or a similar algebraic constraint system. A standard arithmetization of a $k \times k$ matrix product $\mathbf{AB} = \mathbf{C}$ computes every output entry and therefore uses $\mathcal{O}(k^3)$ arithmetic constraints. This is already impractical for moderate k, and it is especially problematic for Transformer-based neural networks [5]–[7], whose projection and attention layers often contain matrix multiplications with dimension $k \geq 1{,}000$.

The $\mathcal{O}(k^2)$ barrier. Freivalds' classical randomized check [8] reduces matrix-product verification from a cubic computation to a quadratic one. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, the verifier checks

$$\mathbf{rAB} = \mathbf{rC}$$

instead of recomputing the full product. This reduces the verification cost from $\mathcal{O}(k^3)$ to $\mathcal{O}(k^2)$. The resulting $\mathcal{O}(k^2)$-size circuit is an improvement, but it is still expensive: for example, at $k = 2^{12}$, it already requires 50.5 million R1CS constraints and $411.28\,\mathrm{s}$ of proving time with Groth16. Thus, a large gap remains between the $\mathcal{O}(k^2)$ SNARK cost and practical large-scale matrix-multiplication verification.

Our insight: from computation to checking. We observe that the $\mathcal{O}(k^2)$ barrier is not inherent to the verification problem itself. It arises because existing SNARK-based approaches still *compute* the vector–matrix products inside the circuit. Our key idea is to move from "computing inside the circuit" to "*checking* inside the circuit." The prover performs the expensive matrix arithmetic outside the SNARK and commits to the relevant encoded data and intermediate values. The verifier then checks only a small number of lightweight algebraic relations that are sufficient to detect inconsistent claims. The technical core of this approach is proximity testing over linear error-correcting codes. Let $\mathcal{C}$ be a linear code with relative distance δ. We encode each matrix row-wise and consider the Freivalds intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}.$$

The linearity of $\mathcal{C}$ lets us check these vector–matrix products in the encoded domain, column by column. For one encoded column, a single inner product, or *fold*, checks one coordinate of the encoded product. If the prover cheats in one of the corresponding relations, the two relevant codewords disagree on at least a δ-fraction of positions. A random query therefore detects the inconsistency with probability at least δ.

The key point is that the SNARK circuit never takes the full matrices as witnesses. Instead, it checks fold equations on t sampled encoded columns, where t is independent of the matrix dimension k. It also checks that the committed encoded views of the intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ are valid codewords for those vectors. To bind the circuit witnesses to the data fixed before the query positions are known, we use a commit-and-prove SNARK for the sampled block witnesses. The prover also supplies Merkle membership proofs for the queried leaves under rt_{ABC} and rt_{XYZ}, together with CP-Link proofs showing that the SNARK-side sampled blocks and the opened Pedersen leaves contain the same data. These checks ensure, under the backend assumptions, that the values checked inside the circuit are consistent with commitments fixed before the verifier's sampling challenges.

As a result, the main SNARK relation has $\mathcal{O}(tk+E_{\mathsf{code}})$ constraints, where E_{code} is the cost of checking that the intermediate vectors are valid encodings under the error-correcting code. For the fixed-rate Reed–Solomon setting used in our implementation, this remains linear in the matrix dimension for fixed t. The sampled opening and linking layer adds $\mathcal{O}(t \log n) + E_{\mathsf{link}}(k, t)$ backend work. In our implementation, verification consists of SNARK verification, CP-Link verification, and $\mathcal{O}(t \log n)$ hash operations for Merkle openings; for the QALink instantiation, CP-Link verification includes $\mathcal{O}(t)$ pairing terms batched into a multi-pairing check.

Contributions. We present LAMP, a protocol for efficiently proving matrix-multiplication statements. Our contributions are as follows.

- **A code-based matrix-product checking protocol.** LAMP combines Freivalds' randomized reduction with proximity testing over linear error-correcting codes. Instead of computing vector–matrix products inside the SNARK circuit, the protocol checks sampled fold equations over encoded commitments and verifies code consistency for the intermediate vectors.
  For fixed t, the main circuit cost is $\mathcal{O}(tk + E_{\mathsf{code}})$, which is linear in k for the fixed-rate Reed–Solomon setting used in our implementation.
- **Soundness analysis.**
  We prove soundness in the committed-input setting by leveraging the relative distance and unique-decoding radius of the Reed–Solomon code. The analysis also accounts for Merkle openings and CP-Link proofs, which bind the values checked inside the SNARK circuit to the data committed before the sampling challenges are derived.
- **Implementation and evaluation.**
  We implement LAMP in Go and evaluate it against a Freivalds-based SNARK baseline and DualMatrix. In the square-matrix experiments with $\rho = 1/2$ and $t = 309$, LAMP reduces the constraint count by $9.68\times$ and proving time by $5.88\times$ relative to the Freivalds baseline at $k = 2^{12}$. It also proves faster than DualMatrix at every matrix dimension measured for both systems, achieving a $3.65\times$ speedup at $k = 2^{13}$.
  We further evaluate batching and a GPT-2 workload. Batching matrix products with a shared leaf layout keeps the Merkle proof size and verification time nearly constant as the number of claims increases. On GPT-2, LAMP proves faster than both the Freivalds baseline and DualMatrix, although heterogeneous matrix dimensions require multiple commitment layouts and increase proof size and verification cost.

2. Related Work

Matrix multiplication.

Pinocchio [9] and Groth16 [1] provide constant-size proofs and constant verifier time, but directly applying them to matrix multiplication requires arithmetizing the entire computation. Thus, verifying the multiplication of two $k \times k$ matrices requires $\mathcal{O}(k^3)$ constraints, resulting in $\mathcal{O}(k^3 \log k)$ prover computation.

To overcome this limitation, several proof systems have studied how to verify matrix multiplication more efficiently without directly arithmetizing the full cubic computation. LegoSNARK [10] extends Groth16 with a commit-and-prove mechanism that ensures consistency between committed values and the SNARK witness. Combined with Freivalds' randomized verification, matrix multiplication can be expressed using $\mathcal{O}(k^2)$ constraints, resulting in $\mathcal{O}(k^2 \log k)$ prover complexity while retaining constant-size proofs and constant verifier time. Thaler's sum-check-based protocol [11] reduces matrix multiplication to checks over low-degree extensions, achieving $\mathcal{O}(k^2)$ prover work. However, the verifier computation still grows linearly with the matrix dimension, which can be costly for succinct verification of large matrices.

More recent works have developed specialized proof systems that verify matrix multiplication without relying on arithmetic circuits. zkMatrix [12] reduces matrix multiplication to four inner-product relations and adapts the techniques of Bulletproofs [13], achieving $\mathcal{O}(k^2)$ prover complexity and $\mathcal{O}(\log k)$ verifier complexity and proof size. DualMatrix [14], a follow-up work by the authors of zkMatrix, improves the prover complexity to $\mathcal{O}(K+k)$, where K denotes the number of non-zero entries. However, for dense matrices with $K = \Theta(k^2)$, its prover complexity remains $\mathcal{O}(k^2)$. zkMaP [15] employs KZG commitments to achieve $\mathcal{O}(k^2)$ prover work together with constant-size proofs and constant verifier time.

While these protocols achieve efficient verification of matrix multiplication, they are specialized to matrix multiplication itself. Therefore, in applications such as deep neural networks, where operations beyond matrix multiplication must be proved using a separate SNARK, an additional linking proof is required to ensure consistency between the witnesses used in the matrix multiplication

proof and the SNARK.
Table 2 compares the computational complexities and proof characteristics of existing proof systems for matrix multiplication, including our approach.

Verifiable AI. Matrix multiplication is also a dominant cost in verifiable machine-learning systems. Systems such as vCNN [16], pvCNN [17], and zkVC [18] optimize verifiable inference for convolutional neural networks by reducing the number of multiplication constraints induced by QAP-based representations. Other systems use sum-check-style techniques. zkCNN [19] proposes a sum-check-based protocol for proving convolutions with linear prover time. In addition, zkGPT [20] and zkLLM [21] introduce protocols specialized for LLM architectures and use sum-check protocols to prove the correctness of linear layers efficiently. Meanwhile, Mystique [22] and zkML [23] use Freivalds' randomized checks.

Code-based proximity testing. LAMP also builds on the use of linear error-correcting codes and proximity tests in succinct proof systems. FRI [24] and related works [25], [26] construct proximity IOPs for Reed–Solomon codewords. Recent work further extends this viewpoint to more general foldable codes [27]. Our proximity layer is closer in spirit to the local codeword tests used in Ligero [28], Brakedown [29], and Blaze [30]. In particular, our use of in-circuit encoding checks is similar in spirit to Orion [31] and related work [32].

3. Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p, and λ denotes the security parameter. We write $\mathsf{negl}(\lambda)$ for any function that is negligible in λ, and PPT for probabilistic polynomial-time.

3.1. Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$, and vectors by bold lowercase letters $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a vector $\mathbf{x}$, we write $\mathbf{x}[i]$ for its i-th entry. For a matrix $\mathbf{M}$, we write $\mathbf{M}[i, j]$ for the entry in the i-th row and j-th column. We denote the i-th row of $\mathbf{M}$ by $\mathbf{M}[i, *]$, and we will often write it simply as $\mathbf{M}[i]$. Similarly, we denote the j-th column of $\mathbf{M}$ by $\mathbf{M}[*, j]$. We write $[n] = \{0, 1, \ldots, n-1\}$. For $t \in \mathbb{N}$, $I \leftarrow\!\!\$\ [n]^t$ denotes the uniform sampling of t indices from $[n]$. Similarly, $r \leftarrow\!\!\$\ \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

3.2. Linear Error-Correcting Codes

A *linear code* over $\mathbb{F}$ is an $\mathbb{F}$-linear map $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ defined by a generator matrix $\mathbf{G} \in \mathbb{F}^{k\times n}$, so that $\mathcal{C}(\mathbf{m}) = \mathbf{m} \cdot \mathbf{G}$ for any input vector $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c}\neq\mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$, where the minimum is taken over distinct codewords $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, and Δ denotes the Hamming distance.

The *relative distance* is $\delta = d/n$. The unique-decoding radius is $\tau_{\mathsf{UD}} = \frac{1}{n}\left\lfloor \frac{d-1}{2} \right\rfloor$, within which a received word has at most one nearby codeword. We use a proximity threshold $\tau \le \tau_{\mathsf{UD}}$ throughout the protocol.

We write $\mathsf{Enc}(\mathbf{m}) := \mathcal{C}(\mathbf{m})$ for the encoding of an input vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k\times k}$, we define the *row-wise encoding* $\widehat{\mathbf{M}} := \mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k\times n}$ as the matrix whose i-th row is $\mathsf{Enc}(\mathbf{M}[i, *])$ for each $i \in [k]$. The j-th column of $\widehat{\mathbf{M}}$ is denoted by $\widehat{\mathbf{M}}[*, j] \in \mathbb{F}^k$ for each $j \in [n]$.

Definition 3.1 (Interleaved code)**.** For an integer $\ell \ge 1$, the ℓ-fold interleaved code $\mathcal{C}^{\langle \ell \rangle}$ is obtained by grouping ℓ codewords of $\mathcal{C}$ position by position. More concretely, for codewords

$$\mathbf{c}^{(0)}, \ldots, \mathbf{c}^{(\ell-1)} \in \mathcal{C},$$

their interleaving is the word $\mathbf{w} \in (\mathbb{F}^\ell)^n$ whose j-th symbol is

$$\mathbf{w}[j] = \left(\mathbf{c}^{(0)}[j], \ldots, \mathbf{c}^{(\ell-1)}[j]\right) \in \mathbb{F}^\ell \quad \text{for } j \in [n].$$

The code $\mathcal{C}^{\langle \ell \rangle}$ is the set of all such interleavings.

The row-wise encoding of a matrix is an interleaved codeword in this sense. Indeed, for $\mathbf{M} \in \mathbb{F}^{k\times k}$, the rows $\mathsf{Enc}(\mathbf{M}[0, *]), \ldots, \mathsf{Enc}(\mathbf{M}[k-1, *])$ are k codewords of $\mathcal{C}$, and their interleaving is exactly $\widehat{\mathbf{M}} \in \mathbb{F}^{k\times n}$. The j-th interleaved symbol is the encoded column $\widehat{\mathbf{M}}[*, j] \in \mathbb{F}^k$.

The key property we exploit is *linearity*: for any coefficients $r_0, \ldots, r_{k-1} \in \mathbb{F}$,

$$\sum_{i=0}^{k-1} r_i \cdot \widehat{\mathbf{M}}[i, *] = \mathsf{Enc}\left(\sum_{i=0}^{k-1} r_i \cdot \mathbf{M}[i, *]\right) \tag{1}$$

Definition 3.2 (Fold)**.** For vectors $\mathbf{c}, \mathbf{r} \in \mathbb{F}^k$ with $\mathbf{c} = (c_0, \ldots, c_{k-1})^\top$ and $\mathbf{r} = (r_0, \ldots, r_{k-1})$, define

$$\mathsf{Fold}(\mathbf{r}, \mathbf{c}) := \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j-th column of a row-wise encoded matrix, $\mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{M}}[*, j])$ equals the j-th coordinate of $\mathsf{Enc}(\mathbf{rM})$. This follows directly from the linearity of the code (Equation 1).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $\mathcal{C}(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [29], which achieve linear-time encoding $\mathcal{O}(n)$ with constant relative distance.

3.3. Commitment Schemes

A commitment scheme allows a committer to bind itself to a message and later open it. Some commitment schemes also hide the message before opening; our soundness arguments use only binding. In this work, we use two types

System	Prover Work	Circuit Constraints	Verifier Work	Proof Size
Groth16	$\mathcal{O}(k^3 \log k)$	$\mathcal{O}(k^3)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
LegoSNARK	$\mathcal{O}(k^2 \log k)$	$\mathcal{O}(k^2)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
zkMatrix	$\mathcal{O}(k^2)$	–	$\mathcal{O}(\log k)$	$\mathcal{O}(\log k)$
DualMatrix	$\mathcal{O}(K + k)$	–	$\mathcal{O}(\log k)$	$\mathcal{O}(\log k)$
zkMaP	$\mathcal{O}(k^2)$	–	$\mathcal{O}(1)$	$\mathcal{O}(1)$
LAMP	$\mathcal{O}(k^2)$	$\mathcal{O}(tk)$	$\mathcal{O}(t \log k)$	$\mathcal{O}(t \log k)$

TABLE 1. Asymptotic comparison with systems for matrix multiplication. Here, $K = \Theta(k^2)$ for dense matrices and t is constant.

of commitments: Merkle tree commitments and Pedersen vector commitments.

3.3.1. Merkle tree commitments. A Merkle tree commitment over a collision-resistant hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$ is a vector commitment for a sequence $\mathbf{v} = (\mathbf{v}[j])_{j\in[n]}$. The leaves may be randomized or deterministic. We write $\mathsf{MT.Com}(\mathbf{v}; \mathbf{o})$ for randomized leaves and $\mathsf{MT.Com}(\mathbf{v}; \perp)$ for deterministic leaves.

- $\mathsf{rt} \leftarrow \mathsf{MT.Com}(\mathbf{v}; \mathbf{o})$: outputs a root hash rt obtained by constructing a binary hash tree with randomized leaves

$$\ell_j = \begin{cases} H(\mathbf{v}[j], \mathbf{o}[j]) & \text{if } \mathbf{o} \neq \perp, \\ H(\mathbf{v}[j]) & \text{if } \mathbf{o} = \perp. \end{cases}$$

  Internal nodes are computed by hashing their two children.
- $(\mathbf{v}[j], o_j, \pi_j) \leftarrow \mathsf{MT.Open}(\mathbf{v}, \mathbf{o}, j)$: outputs the leaf value $\mathbf{v}[j]$, its opening randomness o_j, and its authentication path π_j for position $j \in [n]$. In the deterministic case $\mathbf{o} = \perp$, we omit the randomness and write $(\mathbf{v}[j], \pi_j) \leftarrow \mathsf{MT.Open}(\mathbf{v}, j)$.
- $b \leftarrow \mathsf{MT.Verify}(\mathsf{rt}, j, \mathbf{v}[j], \mathbf{o}[j], \pi_j)$: recomputes the randomized leaf $\ell_j = H(\mathbf{v}[j], \mathbf{o}[j])$ and outputs $b = 1$ if the authentication path is consistent with root rt at position j, and outputs $b = 0$ otherwise. In the deterministic case, we write $\mathsf{MT.Verify}(\mathsf{rt}, j, \mathbf{v}[j], \pi_j)$.

Merkle tree commitments are *position-binding*: under the collision resistance of H, no PPT adversary can produce a root rt, an index $j \in [n]$, two distinct openings $(v, o) \neq (v', o')$, and authentication paths π_j, π'_j such that

$$\mathsf{MT.Verify}(\mathsf{rt}, j, v, o, \pi_j) = \mathsf{MT.Verify}(\mathsf{rt}, j, v', o', \pi'_j) = 1,$$

except with probability $\epsilon_{\mathsf{merkle}}(\lambda) = \mathsf{negl}(\lambda)$.

3.3.2. Pedersen commitments. Pedersen commitments can be viewed as algebraic vector commitments over a group $\mathbb{G}$ of prime order q. Let $\mathsf{ck} = (G_0, \ldots, G_{n-1}, H)$ be a commitment key consisting of fixed generators in $\mathbb{G}$. A Pedersen commitment is defined by the following algorithms.

- $\mathsf{cm} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}, \mathbf{v}; o)$: outputs a commitment $\mathsf{cm} := \sum_{i=0}^{n-1} \mathbf{v}[i]G_i + oH \in \mathbb{G}$ for a vector $\mathbf{v} \in \mathbb{F}^n$ and randomness $o \leftarrow\!\!\$\ \mathbb{F}$.
- $\pi \leftarrow \mathsf{Ped.Open}(\mathbf{v}, o)$: outputs the opening $\pi = (\mathbf{v}, o)$ consisting of the committed vector and the randomness.
- $b \leftarrow \mathsf{Ped.Verify}(\mathsf{ck}, \mathsf{cm}, \mathbf{v}, o)$: outputs $b = 1$ if $\mathsf{cm} \stackrel{?}{=} \sum_{i=0}^{n-1} \mathbf{v}[i]G_i + oH$, and outputs $b = 0$ otherwise.

Pedersen commitments are *perfectly hiding* and *computationally binding*: under the discrete logarithm assumption, no PPT adversary can produce a commitment cm, two distinct vectors $\mathbf{v} \neq \mathbf{v}'$, and randomness values o, o' such that

$$\mathsf{Ped.Verify}(\mathsf{ck}, \mathsf{cm}, \mathbf{v}, o) = \mathsf{Ped.Verify}(\mathsf{ck}, \mathsf{cm}, \mathbf{v}', o') = 1,$$

except with probability $\epsilon_{\mathsf{ped}}(\lambda) = \mathsf{negl}(\lambda)$.

They are also *additively homomorphic*: for $\mathsf{cm}_1 = \mathsf{Ped.Com}(\mathsf{ck}, \mathbf{v}_1; o_1)$ and $\mathsf{cm}_2 = \mathsf{Ped.Com}(\mathsf{ck}, \mathbf{v}_2; o_2)$, $\mathsf{cm}_1 + \mathsf{cm}_2 = \mathsf{Ped.Com}(\mathsf{ck}, \mathbf{v}_1 + \mathbf{v}_2; o_1 + o_2)$.

3.4. Succinct Non-interactive Arguments of Knowledge

A SNARK (succinct non-interactive argument of knowledge) for an NP relation $\mathcal{R}$ is a tuple of algorithms $\mathbf{\Pi} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $\mathsf{pp} \leftarrow \mathbf{\Pi}.\mathsf{Setup}(1^\lambda, \mathcal{R})$: generates public parameters pp for the relation $\mathcal{R}$.
- $\pi \leftarrow \mathbf{\Pi}.\mathsf{Prove}(\mathsf{pp}, \mathsf{x}, \mathsf{w})$: outputs a proof π for a statement x and witness w satisfying $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$.
- $b \leftarrow \mathbf{\Pi}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \pi)$: outputs $b = 1$ if π is accepted as a valid proof for x, and outputs $b = 0$ otherwise.

Remark 1 (Security Properties). We use the standard SNARK notions of completeness, knowledge soundness, and zero knowledge. Appendix A recalls these notions formally, including their negligible error bounds.

3.4.1. Commit-and-Prove SNARKs. A commit-and-prove SNARK (CP-SNARK) [10], [33] is a variant of a SNARK that proves statements about witnesses bound to an external commitment. We call such a commitment a *witness commitment*. Let Com be a commitment scheme with commitment key ck. For an NP relation $\mathcal{R}$ over tuples $(\mathsf{x}, \mathsf{u}, \omega)$, a CP-SNARK proves knowledge of a witness $\mathsf{w} := (\mathsf{u}, \omega)$ such that $(\mathsf{x}, \mathsf{u}, \omega) \in \mathcal{R}$, where u is the committed part of the witness and ω is the remaining private witness. The committed witness is bound separately by a witness commitment $\widetilde{\mathsf{cm}} = \mathsf{Com}(\mathsf{ck}_S, m_0 || ... || m_{q-1}; \tilde{o})$, where $\tilde{o}$ is the opening randomness. Equivalently, the CP proof is verified with respect to the extended public statement $\mathsf{x}_{\mathsf{cp}} := (\mathsf{x}, \widetilde{\mathsf{cm}})$. When the context is clear, we write this extended statement simply as x. A CP-SNARK scheme is a tuple of algorithms $\mathbf{\Pi}_{\mathsf{cp}} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{Prove}, \mathsf{Verify})$.

- $\mathsf{pp} \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Setup}(1^\lambda, \mathcal{R})$: generates public parameters $\mathsf{pp} = (\mathsf{pk}, \mathsf{vk}, \mathsf{ck})$ for the relation $\mathcal{R}$, where pk is the proving key, vk is the verifying key, and ck is the commitment key.
- $\widetilde{\mathsf{cm}} \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}(\mathsf{pp}, \mathsf{u}; \tilde{o})$: outputs a witness commitment to the committed witness part u.
- $\pi_{\mathsf{cp}} \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pp}, \mathsf{x}_{\mathsf{cp}}, \mathsf{w})$: outputs a proof for the extended statement x_{cp} and witness $\mathsf{w} = (\mathsf{u}, \omega)$.
- $b \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}_{\mathsf{cp}}, \pi_{\mathsf{cp}})$: outputs $b = 1$ if π_{cp} is accepted as a valid proof for the extended statement x_{cp}, and outputs $b = 0$ otherwise. Equivalently, we may write verification as $\mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \widetilde{\mathsf{cm}}, \pi_{\mathsf{cp}})$.

3.5. CP-Link

A CP-Link proof is a proof system for linking commitments. In the form used by LAMP, it proves that one SNARK-side commitment to an ordered concatenation of sampled blocks is consistent with external commitments to the individual blocks, without revealing the blocks or the opening randomness.

Let Com be a commitment scheme. For a SNARK-side key ck_S, an external key ck_E, a SNARK-side commitment $\widetilde{\mathsf{cm}}$, and external commitments $\mathsf{cm}_0, \ldots, \mathsf{cm}_{q-1}$, the blockwise linking relation is defined over the statement and witness

$$\mathsf{x}_{\mathsf{link}} = ((\mathsf{ck}_S, \widetilde{\mathsf{cm}}), (\mathsf{ck}_E, \mathsf{cm}_0, \ldots, \mathsf{cm}_{q-1})),$$
$$\mathsf{w}_{\mathsf{link}} = ((\mathbf{m}_i)_{i=0}^{q-1}, \widetilde{o}, (o_i)_{i=0}^{q-1}).$$

The pair $(\mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}})$ is in $\mathcal{R}_{\mathsf{link}}$ if and only if

$$\widetilde{\mathsf{cm}} = \mathsf{Com}(\mathsf{ck}_S, \mathbf{m}_0 \| \cdots \| \mathbf{m}_{q-1}; \widetilde{o}),$$
$$\mathsf{cm}_i = \mathsf{Com}(\mathsf{ck}_E, \mathbf{m}_i; o_i) \quad \text{for all } i = 0, \ldots, q-1.$$

A proof for $\mathcal{R}_{\mathsf{link}}$ convinces the verifier that the ordered blocks committed inside the CP-SNARK are exactly the blocks committed by the corresponding external commitments.

We use CP-Link through the standard statement/witness interface for a linking relation.

A CP-Link proof system is a tuple of algorithms $\mathbf{\Pi}_{\mathsf{link}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $\mathsf{pp}_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{link}})$: generates public parameters for the linking relation.
- $\pi_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}})$: outputs a proof that the statement-witness pair satisfies the linking relation.
- $b \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \pi_{\mathsf{link}})$: outputs $b = 1$ if the proof is accepted, and outputs $b = 0$ otherwise.

Remark 2 (Security Properties). For the soundness of LAMP, we require CP-Link soundness: an accepting linking proof should imply membership in $\mathcal{R}_{\mathsf{link}}$, except with error $\epsilon_{\mathsf{link}}(\lambda)$. If the linking backend is used to protect the linked blocks and openings, one may additionally require witness zero knowledge. Appendix B states these properties.

3.6. Freivalds' Algorithm

Freivalds' algorithm [8] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$, it samples $\mathbf{r} \leftarrow\!\!\$\ \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

Lemma 3.3 (Schwartz–Zippel [34], [35])**.** *Let $P(X_1, \ldots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \ldots, r_m \leftarrow\!\!\$\ \mathbb{F}$ chosen uniformly and independently,*

$$\Pr[P(r_1, \ldots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow\!\!\$\ \mathbb{F}^k$ uniformly, we sample one scalar $r \leftarrow\!\!\$\ \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$. This keeps the Freivalds challenge compact in the transcript and in the SNARK statement: the verifier records one field element rather than k independent ones. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the value $\langle \mathbf{r}, \mathbf{d}_j \rangle$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 3.3, it vanishes with probability at most $(k-1)/|\mathbb{F}|$.

4. Overview of LAMP

This section gives a technical overview of LAMP before the formal construction in Section 5. The main message is that LAMP does not ask the SNARK circuit to compute a matrix product. Instead, the prover commits to encoded data, and the circuit checks only a small number of local consistency equations at randomly sampled codeword positions.

4.1. Problem Statement and Relation Reduction Chain

The starting point is the matrix multiplication relation

$$\mathbf{AB} = \mathbf{C}, \qquad \mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}.$$

A direct SNARK arithmetization of this relation computes every entry of $\mathbf{AB}$. This requires k^3 scalar multiplications and therefore $\mathcal{O}(k^3)$ arithmetic constraints. This is the original relation:

$$\mathcal{R}_{\mathsf{orig}} = \{(\mathbf{A}, \mathbf{B}, \mathbf{C}) : \mathbf{AB} = \mathbf{C}\}.$$

Freivalds' algorithm reduces the cubic check to a vector-matrix check. The classical algorithm samples a challenge vector $\mathbf{r} \in \mathbb{F}^k$ and checks $\mathbf{rAB} = \mathbf{rC}$.

In LAMP, the verifier derives a scalar $r \in \mathbb{F}$ and sets $\mathbf{r} = (1, r, \ldots, r^{k-1})$, which keeps the public challenge compact. At the same transcript point, the verifier derives an independent scalar $s \in \mathbb{F}$ and sets $\mathbf{s} = (1, s, \ldots, s^{k-1})$. The role of this additional challenge in testing the proximity of the committed encoding of $\mathbf{B}$ is described in Section 4.2

Equivalently, the prover introduces intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC},$$

and the verifier checks $\mathbf{y} = \mathbf{z}$. This gives the Freivalds relation

$$\mathcal{R}_{\mathsf{Fr}} = \Big\{(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}) : \quad \mathbf{x} = \mathbf{r}\mathbf{A}, \quad \mathbf{y} = \mathbf{x}\mathbf{B}, \quad \mathbf{z} = \mathbf{r}\mathbf{C}, \quad \mathbf{y} = \mathbf{z}\Big\}.$$

This removes the cubic matrix-matrix computation, but a SNARK circuit that computes $\mathbf{rA}$, $\mathbf{xB}$, and $\mathbf{rC}$ directly still performs $\mathcal{O}(k^2)$ scalar multiplications.

LAMP applies one more reduction. Rather than computing the full vector-matrix products inside the circuit, it checks the products only at randomly sampled positions in an encoded domain. The prover performs the expensive native computation outside the SNARK, commits to the encoded matrices and intermediate vectors, and the SNARK checks local equations of the form

$$\begin{aligned} \mathsf{Enc}(\mathbf{x})[j] &= \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]), \\ \mathsf{Enc}(\mathbf{y})[j] &= \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]), \\ \mathsf{Enc}(\mathbf{z})[j] &= \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]). \end{aligned}$$

The prover additionally computes $\mathbf{b} = \mathbf{sB}$, the circuit checks

$$\mathsf{Enc}(\mathbf{b})[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j]).$$

An additional fold based on the independent challenge $\mathbf{s}$ is used to test the proximity of the committed encoding of $\mathbf{B}$, as described in Section 4.2.

Only the queried columns $\widehat{\mathbf{A}}[*, j]$, $\widehat{\mathbf{B}}[*, j]$, $\widehat{\mathbf{C}}[*, j]$ are used by the circuit. Thus, for t queried positions, the fold part of the circuit has cost $\mathcal{O}(tk)$, which is linear in k for fixed t.

4.2. Encoding and Proximity Testing

We use the proximity threshold $\tau \leq \tau_{\mathsf{UD}}$ defined in Section 3.2. For a committed row-wise encoding, proximity is measured with respect to the corresponding interleaved code $\mathcal{C}^{\langle k \rangle}$.

For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, the prover row-wise encodes $\mathbf{M}$ as $\widehat{\mathbf{M}} = \mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$. By Definition 3.1, this row-wise encoding can also be viewed as a k-fold interleaved codeword, where the j-th interleaved symbol is the encoded column $\widehat{\mathbf{M}}[*, j]$.

The key property underlying the local checks is the linearity of the code. For any $\mathbf{u} = (u_0, \ldots, u_{k-1}) \in \mathbb{F}^k$ and every $j \in [n]$,

$$\mathsf{Enc}(\mathbf{uM})[j] = \sum_{i=0}^{k-1} u_i \widehat{\mathbf{M}}[i, j] = \mathsf{Fold}\big(\mathbf{u}, \widehat{\mathbf{M}}[*, j]\big).$$

Thus, each coordinate of an encoded vector–matrix product can be checked using only the corresponding encoded column. Applying this identity to the Freivalds intermediates gives, for every $j \in [n]$, $\mathbf{x} = \mathbf{rA}$, then for every $j \in [n]$,

$$\widehat{\mathbf{x}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]), \qquad \widehat{\mathbf{z}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]),$$

and

$$\widehat{\mathbf{y}}[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]).$$

The last equation alone, however, does not provide a proximity test for the committed encoding $\widehat{\mathbf{B}}$. The folding vector $\mathbf{x} = \mathbf{rA}$ depends on $\mathbf{A}$ and can be degenerate. For example, if $\mathbf{A} = 0$, then $\mathbf{x} = 0$ for every challenge r, and the resulting $\mathbf{x}$-fold reveals no information about whether $\widehat{\mathbf{B}}$ is close to a valid interleaved codeword. To obtain an independent proximity test for $\widehat{\mathbf{B}}$, the prover computes $\mathbf{b} = \mathbf{sB}$. By the same linearity property, $\widehat{\mathbf{b}}[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j])$ for every $j \in [n]$. Thus, the $\mathbf{r}$-fold for $\mathbf{A}, \mathbf{C}$ and the $\mathbf{s}$-fold for $\mathbf{B}$ tests the proximity of $\widehat{\mathbf{A}}, \widehat{\mathbf{C}}$ and $\widehat{\mathbf{B}}$, whereas the $\mathbf{x}$-fold checks consistency of the claimed relation $\mathbf{y} = \mathbf{xB}$.

4.3. Sampled-Opening Layer

The sampled checks above are meaningful only if the values used inside the SNARK are bound to data fixed before the relevant challenges are sampled. LAMP achieves this through a sampled-opening layer. The prover first commits to the interleaved symbols, equivalently the encoded matrix columns, and fixes a Merkle root before the Freivalds challenge is derived. After the challenge is fixed, the prover computes the intermediate vectors, commits to their encoded symbols, and fixes a second Merkle root before the sampled positions are derived.

The SNARK circuit itself checks field equations: the sampled fold and equality checks, together with the code-consistency checks for the intermediate vectors. It does not verify Pedersen openings or Merkle paths internally. Instead, the commit-and-prove SNARK commits to the sampled witness blocks used by the circuit, and the prover supplies:

- Merkle openings showing that the external Pedersen commitments $\mathsf{cm}_{ABC,j}$ and $\mathsf{cm}_{XYZ,j}$ are authenticated under rt_{ABC} and rt_{XYZ};
- CP-Link proofs showing that the witness commitments from CP-SNARK and the external Pedersen commitments open to the corresponding sampled blocks.

Thus the verifier obtains a chain of consistency:

SNARK witness values

$\Updownarrow$ CP-Link

External Pedersen leaf commitments

$\Updownarrow$ Merkle openings

Fixed Merkle roots

This separation is essential for efficiency. The SNARK checks only arithmetic relations on field elements, while

Merkle authentication and commitment linking are verified outside the circuit.

4.4. Soundness Intuition

The soundness argument has three layers.

Freivalds soundness. If $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{D} = \mathbf{AB} - \mathbf{C}$ is nonzero. With the structured challenge $\mathbf{r} = (1, r, \ldots, r^{k-1})$, the vector $\mathbf{rD}$ is nonzero except with probability at most $(k-1)/|\mathbb{F}|$, by Schwartz–Zippel. Therefore, except with this probability, at least one of the vector relations in the Freivalds reduction is false.

Proximity soundness.

The committed matrix encodings need not be exact codewords. If a committed encoding is τ-far from the interleaved code, the proximity-gap [36] property ensures, except with a small failure probability over the folding challenge, that its folded word remains τ-far from every valid codeword. Otherwise, if it is τ-close to an underlying codeword but the prover claims an inconsistent vector relation, the code distance δ guarantees that the folded word and the claimed encoding differ on at least a $(\delta - \tau)$-fraction of positions. Since $\tau \leq \tau_{\mathsf{UD}}$ implies $\delta - \tau \geq \tau$, an invalid claim leaves at least a τ-fraction of inconsistent positions in either case. Therefore, t random queries miss all inconsistencies with probability at most $(1-\tau)^t$. The proximity-gap failure over the folding challenges is accounted for separately in the formal soundness analysis 6.

Commitment and linking soundness. The distance argument applies only to values fixed before the query positions are sampled. Merkle position binding prevents the prover from opening a root to two different leaves at the same position. Pedersen binding prevents a committed leaf from being opened to two different messages. CP-Link ensures that the values used by the SNARK are the same values authenticated by the external commitments. Therefore, except for the binding and linking failure probabilities, any accepting proof corresponds to fixed encoded objects that pass the sampled proximity checks.

Putting the layers together, a cheating prover must either break one of the commitment/linking mechanisms, violate SNARK soundness, hit the Freivalds failure event, or avoid all sampled disagreements in the encoded domain. The formal theorem in Section 6 makes this union bound explicit.

5. The LAMP Protocol

Section 4 reduced the original matrix-multiplication relation to the Freivalds relation and then to sampled encoded checks. This section formalizes the relation checked by LAMP. The important point is that LAMP is not a single arithmetic-circuit relation: the arithmetic checks are proved by an inner CP-SNARK, while Merkle membership and CP-Link are verified outside the circuit.

5.1. The Composite LAMP Relation

Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, and let $\mathcal{C} : \mathbb{F}^k \to \mathbb{F}^n$ be a linear code with relative distance δ. The target matrix-multiplication relation is

$$\mathcal{R}_{\mathsf{orig}} = \{(\mathbf{A}, \mathbf{B}, \mathbf{C}) : \mathbf{AB} = \mathbf{C}\}.$$

The verifier does not receive the original matrices as public input. Instead, the prover commits to row-wise encodings of the matrices and proves that the committed objects satisfy the multiplication relation.

The relation checked by LAMP is the composite relation

$$\mathcal{R}_{\mathsf{LAMP}} = \mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}} \wedge \mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}.$$

Here $\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}$ is the online relation proved by the CP-SNARK after the Freivalds challenge, query positions, and code-check points are sampled. The relation $\mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}$ is the offline relation: it captures the values fixed by commitments before the online queries are known, and the verifier checks it through Merkle openings and CP-Link proofs. Sections 5.3 and 5.4 define these two relations explicitly. The circuit checks field equations on sampled encoded values, while the auxiliary verifier checks that those values are the ones bound by the pre-query commitments.

5.2. Commitment Layout and Public Challenges

The prover first encodes the matrices row-wise: $\widehat{\mathbf{A}} = \mathsf{Enc}(\mathbf{A}), \widehat{\mathbf{B}} = \mathsf{Enc}(\mathbf{B}), \widehat{\mathbf{C}} = \mathsf{Enc}(\mathbf{C})$. For each codeword position $j \in [n]$, it packs the three k-dimensional interleaved symbols, equivalently the three encoded columns, into $\mathbf{m}_{ABC,j} := (\widehat{\mathbf{A}}[*, j] \,\|\, \widehat{\mathbf{B}}[*, j] \,\|\, \widehat{\mathbf{C}}[*, j]) \in \mathbb{F}^{3k}$. Using a Pedersen commitment key ck_{ABC}, the prover samples $o_{ABC,j} \leftarrow\$\ \mathbb{F}$ and computes $\mathsf{cm}_{ABC,j} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j})$. The matrix root is the Merkle commitment to these Pedersen commitments: $\mathsf{rt}_{ABC} \leftarrow \mathsf{MT.Com}((\mathsf{cm}_{ABC,j})_{j \in [n]}; \perp)$. Here and below, $\perp$ indicates that no additional randomness is used in the Merkle tree, since the leaves are Pedersen commitments that already provide hiding. The root rt_{ABC} is fixed before the Freivalds challenge is derived.

After receiving rt_{ABC}, the verifier samples r, $s \leftarrow\$\ \mathbb{F}$. The prover sets $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$, $\mathbf{s} = (1, s, \ldots, s^{k-1})$, and computes $\mathbf{x} = \mathbf{rA}, \mathbf{y} = \mathbf{xB}, \mathbf{z} = \mathbf{rC}$, $\mathbf{b} = \mathbf{sB}$. It then encodes the intermediate vectors: $\widehat{\mathbf{x}} = \mathsf{Enc}(\mathbf{x}), \widehat{\mathbf{y}} = \mathsf{Enc}(\mathbf{y}), \widehat{\mathbf{z}} = \mathsf{Enc}(\mathbf{z})$, $\widehat{\mathbf{b}} = \mathsf{Enc}(\mathbf{b})$. The encoded intermediate vectors form a 4-fold interleaved codeword. For each $j \in [n]$, the prover packs its j-th interleaved symbol into $\mathbf{m}_{XYZ,j} := (\widehat{\mathbf{x}}[j] \,\|\, \widehat{\mathbf{y}}[j] \,\|\, \widehat{\mathbf{z}}[j] \,\|\, \widehat{\mathbf{b}}[j]) \in \mathbb{F}^{4}$. Using a Pedersen commitment key ck_{XYZ}, the prover samples $o_{XYZ,j} \leftarrow\$\ \mathbb{F}$, computes $\mathsf{cm}_{XYZ,j} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{XYZ}, \mathbf{m}_{XYZ,j}; o_{XYZ,j})$, and sends the vector-symbol root $\mathsf{rt}_{XYZ} \leftarrow \mathsf{MT.Com}((\mathsf{cm}_{XYZ,j})_{j \in [n]}; \perp)$ to the verifier.

Only after receiving rt_{XYZ} does the verifier sample the query tuple $I = (j_1, \ldots, j_t) \leftarrow\$\ [n]^t$. It also samples public code-check points $\alpha_x, \alpha_y, \alpha_z$, α_b $\leftarrow\$\ \mathbb{F} \setminus D$, where D is the code evaluation domain. These points are used by the circuit to check that $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ are encodings of the claimed intermediate vectors. For the Reed–Solomon code used in our implementation, this check is the barycentric out-of-domain consistency test recalled in Appendix D.

5.3. The Online Relation

The online relation is the arithmetic relation proved inside the inner CP-SNARK. Its role is not to verify Merkle paths or Pedersen openings. Instead, it checks only the field equations that certify the sampled encoded positions are consistent with the Freivalds reduction.

The public statement contains the Merkle roots and verifier-sampled challenges:

$$\mathsf{x} = \left(\mathsf{rt}_{ABC}, \mathsf{rt}_{XYZ}, r, I, \alpha_x, \alpha_y, \alpha_z, \alpha_b\right).$$

The roots are included in the statement to bind the CP-SNARK proof to the same transcript and offline checks; the online field equations themselves do not evaluate Merkle paths.

The complete SNARK witness is $\mathsf{w} = (\mathsf{u}, \omega)$, where the committed part of the witness is

$$\mathsf{u} = \left((\mathbf{m}_{ABC,j})_{j \in I}, (\mathbf{m}_{XYZ,j})_{j \in I}\right),$$

and the remaining private witness is $\omega = \left(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}, \widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}\right)$. The CP-SNARK treats u as the committed part of the witness. The prover first computes the SNARK-side witness commitments $\widetilde{\mathsf{cm}}_{ABC}$ and $\widetilde{\mathsf{cm}}_{XYZ}$ using $\mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}$, and then proves the online relation with witness $\mathsf{w} = (\mathsf{u}, \omega)$. These witness commitments are not checked against the Merkle roots inside the circuit. Instead, they are connected to the externally committed Merkle leaves by the offline relation in Section 5.4. For the CP-Link statements below, let $\mathsf{ck}_{\mathsf{cp},T}$ denote the public commitment key used by the CP-SNARK witness commitment $\widetilde{\mathsf{cm}}_T$, for $T \in \{ABC, XYZ\}$. Write $\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1$ when the out-of-domain code-consistency check accepts that $\widehat{\mathbf{v}}$ is consistent with $\mathsf{Enc}(\mathbf{v})$ at the public point α. The online relation is defined in Figure 1.

○ $\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}(\mathsf{x}; \mathsf{w})$:

parse

$\mathsf{x} = \left(\mathsf{rt}_{ABC}, \mathsf{rt}_{XYZ}, r, s, I, \alpha_x, \alpha_y, \alpha_z, \alpha_b\right)$

$\mathsf{w} = (\mathsf{u}, \omega)$

$\mathsf{u} = \left((\mathbf{m}_{ABC,j})_{j \in I}, (\mathbf{m}_{XYZ,j})_{j \in I}\right)$

$\omega = \left(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}, \widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}\right)$

$\mathbf{r} \leftarrow (1, r, \ldots, r^{k-1})$

$\mathbf{s} \leftarrow (1, s, \ldots, s^{k-1})$

$\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{x}}, \mathbf{x}; \alpha_x) = 1$

$\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{y}}, \mathbf{y}; \alpha_y) = 1$

$\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{z}}, \mathbf{z}; \alpha_z) = 1$

$\mathsf{EncCheck}_{\mathcal{C}}(\widehat{\mathbf{b}}, \mathbf{b}; \alpha_b) = 1$

$\forall j \in I$:

parse $\mathbf{m}_{ABC,j}$ as $\left(\widehat{\mathbf{A}}[*, j] \,\|\, \widehat{\mathbf{B}}[*, j] \,\|\, \widehat{\mathbf{C}}[*, j]\right)$

parse $\mathbf{m}_{XYZ,j}$ as $\left(\widehat{\mathbf{x}}[j] \,\|\, \widehat{\mathbf{y}}[j] \,\|\, \widehat{\mathbf{z}}[j] \,\|\, \widehat{\mathbf{b}}[j]\right)$

$\widehat{\mathbf{x}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j])$

$\widehat{\mathbf{y}}[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j])$

$\widehat{\mathbf{z}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j])$

$\widehat{\mathbf{b}}[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j])$

$\widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j]$

Figure 1. The Online Relation

The first four checks certify that $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ are valid encodings of the claimed intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{b}$. The remaining checks are local equations on the sampled codeword positions. For each queried position j, the circuit unpacks the sampled matrix-column block $\mathbf{m}_{ABC,j}$ and the sampled vector-symbol block $\mathbf{m}_{XYZ,j}$, and then verifies the four fold relations together with the final equality $\widehat{\mathbf{y}}[j] = \widehat{\mathbf{z}}[j]$.

Thus, the online relation only checks arithmetic consistency of the sampled values. It does not by itself guarantee that these sampled values came from the pre-query Merkle roots rt_{ABC} and rt_{XYZ}. That binding is provided by the offline relation in Section 5.4.

5.4. The Offline Relation

The offline relation connects the sampled witnesses used by the CP-SNARK to the commitments that were fixed before the query positions were sampled. The term “offline” indicates that these checks are performed outside the SNARK circuit. In our system, it also reflects that the committed data are fixed before the online sampled checks are carried out.

For each queried position $j \in I$, the verifier receives Merkle openings for the external Pedersen leaves $\mathsf{cm}_{ABC,j}$

$$
\begin{aligned}
&\circ\ \mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}\big(\ \mathsf{rt}_{ABC}, \mathsf{rt}_{XYZ}, I, \widetilde{\mathsf{cm}}_{ABC}, \widetilde{\mathsf{cm}}_{XYZ},\\
&\quad \pi^{ABC}_{\mathsf{link}}, \pi^{XYZ}_{\mathsf{link}},\\
&\quad (\mathsf{cm}_{ABC,j}, \pi^{ABC}_{\mathsf{mem},j})_{j\in I}, (\mathsf{cm}_{XYZ,j}, \pi^{XYZ}_{\mathsf{mem},j})_{j\in I}\big) :\\
&\quad \forall j \in I :\\
&\qquad \mathsf{MT.Verify}(\mathsf{rt}_{ABC}, j, \mathsf{cm}_{ABC,j}, \pi^{ABC}_{\mathsf{mem},j}) = 1\\
&\qquad \mathsf{MT.Verify}(\mathsf{rt}_{XYZ}, j, \mathsf{cm}_{XYZ,j}, \pi^{XYZ}_{\mathsf{mem},j}) = 1\\
&\quad \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^{ABC}_{\mathsf{link}}, \pi^{ABC}_{\mathsf{link}}) = 1\\
&\quad \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^{XYZ}_{\mathsf{link}}, \pi^{XYZ}_{\mathsf{link}}) = 1
\end{aligned}
$$

Figure 2. The Offline Relation

and $\mathsf{cm}_{XYZ,j}$. The Merkle openings certify that these leaves are contained in the roots rt_{ABC} and rt_{XYZ}, respectively. In addition, the verifier checks CP-Link proofs connecting the CP-SNARK witness commitments $\widetilde{\mathsf{cm}}_{ABC}$ and $\widetilde{\mathsf{cm}}_{XYZ}$ to the corresponding external Pedersen leaves.

The CP-Link relation is interpreted blockwise. Each block $\mathbf{m}_{T,j}$ is the serialized interleaved symbol opened at position j for the corresponding committed object T. For $T \in \{ABC, XYZ\}$, define the ordered sampled message vector

$$\mathbf{M}_{T,I} := \big(\mathbf{m}_{T,j_1} \,\|\, \mathbf{m}_{T,j_2} \,\|\, \cdots \,\|\, \mathbf{m}_{T,j_t}\big), \quad I = (j_1, \ldots, j_t).$$

The SNARK-side witness commitment $\widetilde{\mathsf{cm}}_T$ commits to the concatenated sampled witness $\mathbf{M}_{T,I}$, while each external Pedersen leaf cm_{T,j_ℓ} commits to the corresponding block $\mathbf{m}_{T,j_\ell}$.

For $T \in \{ABC, XYZ\}$, let $\mathsf{ck}_{\mathsf{cp},T}$ denote the public commitment key used for the CP-SNARK witness commitment $\widetilde{\mathsf{cm}}_T$, and let ck_T denote the external Pedersen commitment key used for Merkle leaves of type T. Using this notation, Figure 2 writes the public CP-Link statement compactly as

$$\mathsf{x}^T_{\mathsf{link}} := \big((\mathsf{ck}_{\mathsf{cp},T}, \widetilde{\mathsf{cm}}_T), (\mathsf{ck}_T, \mathsf{cm}_{T,j_\ell})_{\ell\in[t]}\big).$$

Therefore, the CP-Link proof for T proves the existence of openings such that $\widetilde{\mathsf{cm}}_T = \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}(\mathsf{pp}, \mathbf{M}_{T,I}; \widetilde{o}_T)$, and, for every $\ell \in [t]$, $\mathsf{cm}_{T,j_\ell} = \mathsf{Ped.Com}\big(\mathsf{ck}_T, \mathbf{m}_{T,j_\ell}; o_{T,j_\ell}\big)$. This blockwise interpretation is important: the CP-Link proof claims that the ordered blocks committed inside the CP-SNARK are exactly the same as the ordered blocks committed by the corresponding external Pedersen leaves.

The offline relation is defined in Figure 2.

By combining the Merkle membership checks with the CP-Link checks, we obtain the consistency chain described in Section 4.3. For each $T \in \{ABC, XYZ\}$,

$$
\begin{array}{ccc}
\widetilde{\mathsf{cm}}_T & \xleftrightarrow{\pi^T_{\mathsf{link}}} & (\mathsf{cm}_{T,j})_{j\in I}\\
(\mathsf{cm}_{T,j})_{j\in I} & \xleftrightarrow{(\pi^T_{\mathsf{mem},j})_{j\in I}} & \mathsf{rt}_T
\end{array}
$$

Here $T = ABC$ uses the packed sampled matrix-column messages $(\mathbf{m}_{ABC,j})_{j\in I}$ and the root rt_{ABC}, while $T = XYZ$ uses the packed sampled vector-symbol messages $(\mathbf{m}_{XYZ,j})_{j\in I}$ and the root rt_{XYZ}.

The upper line states that the SNARK-side committed witness and the opened external Pedersen leaves contain the same ordered sampled blocks. The lower line states that those external leaves are authenticated under the corresponding pre-query Merkle root. Therefore, once both $\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}$ and $\mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}$ hold, the verifier knows that the arithmetic checks performed inside the CP-SNARK were applied to values already bound by rt_{ABC} and rt_{XYZ}. This separation is what allows the SNARK circuit to avoid in-circuit Pedersen verification and in-circuit Merkle path verification.

5.5. Protocol Algorithms and Complexity

Figure 3 presents the public-coin version of LAMP. The protocol can be made non-interactive via the Fiat–Shamir transform, by deriving r , s after rt_{ABC} and deriving $I, \alpha_x, \alpha_y, \alpha_z$, α_b after rt_{XYZ}.

The proof consists of the CP-SNARK proof, the sampled leaf commitments and Merkle openings, and the CP-Link proofs. The sampled matrix-column blocks and vector-symbol blocks are private CP-SNARK witnesses; they are not serialized as public proof material. The public statement contains the roots $\mathsf{rt}_{ABC}, \mathsf{rt}_{XYZ}$ and the verifier-sampled challenges.

For every $T \in \mathcal{T} = \{ABC, XYZ\}$, Figure 3 uses the following CP-Link statement and witness:

$$
\begin{aligned}
\mathsf{x}^T_{\mathsf{link}} &:= \big((\mathsf{ck}_{\mathsf{cp},T}, \widetilde{\mathsf{cm}}_T), (\mathsf{ck}_T, \mathsf{cm}_{T,j})_{j\in I}\big),\\
\mathsf{w}^T_{\mathsf{link}} &:= \big((\mathbf{m}_{T,j})_{j\in I}, \widetilde{o}_T, (o_{T,j})_{j\in I}\big).
\end{aligned}
$$

Thus the CP-Link API is used with the public linking parameters, the commitment keys and commitments as the statement, and the sampled messages together with their opening randomness as the witness.

Cost model. We report the cost of LAMP. The computation of the claimed output matrix $C = AB$ is outside this cost model. We do count the work performed by the LAMP prover after the matrices are available, including encoding, commitment generation, computation of the Freivalds intermediate witnesses, sampled Merkle openings, CP-SNARK proving, and CP-Link proving.

Let $E_{\mathsf{Enc}}(k, n)$ denote the cost of the corresponding encoding phase. Let $E_{\mathsf{code}} = E_{\mathsf{code}}(k, n)$ denote the constraint cost of the intermediate code-consistency checks; for the Reed–Solomon check in Appendix D, this is $\mathcal{O}(k + n)$ after domain-dependent coefficients are precomputed. We write $\mathbb{G}$ for group operations used by Pedersen commitments, $\mathbb{F}$ for field operations, and $\mathbb{H}$ for hash evaluations. Let $E^{\mathcal{P}}_{\mathsf{snark}}$ and $E^{\mathcal{V}}_{\mathsf{snark}}$ denote the CP-SNARK proving and verification costs, respectively. Let $E^{\mathcal{P}}_{\mathsf{link}}(k, t)$ and $E^{\mathcal{V}}_{\mathsf{link}}(k, t)$ denote the prover and verifier costs, respectively, of the selected CP-Link construction. For the QALink instantiation in Appendix E, each of the two link statements has $q = t$

external commitments, so $E^{\mathcal{V}}_{\mathsf{link}}(k,t)$ includes $2(t+2)$ Miller-loop inputs, batched into one final exponentiation, plus $\mathcal{O}(t)$ group scalar multiplications for batching.

Table 2 summarizes the asymptotic costs at the level of the main protocol components.

Component	Prover	Verifier
ABC encoding	$E_{\mathsf{Enc}}(k,n)$	–
ABC commit	$\mathcal{O}(3nk)\,\mathbb{G}+\mathcal{O}(n)\,\mathbb{H}$	–
Intermediate	$\mathcal{O}(k^2)\,\mathbb{F}$	–
Intermediate encoding	$E_{\mathsf{Enc}}(k,n)$	–
Intermediate commit	$\mathcal{O}(n)\,\mathbb{G}+\mathcal{O}(n)\,\mathbb{H}$	–
Merkle openings	$\mathcal{O}(t\log n)\mathbb{H}$	$\mathcal{O}(t\log n)\mathbb{H}$
CP-SNARK	$\mathcal{O}(tk)+E_{\mathsf{code}}$ constraints $E^{\mathcal{P}}_{\mathsf{snark}}$	$E^{\mathcal{V}}_{\mathsf{snark}}$
CP-Link	$E^{\mathcal{P}}_{\mathsf{link}}(k,t)$	$E^{\mathcal{V}}_{\mathsf{link}}(k,t)$

TABLE 2. ASYMPTOTIC COSTS FOR LAMP.

Prover work. The prover first encodes the three input matrices and commits to the packed encoded matrix columns. The ABC commitment phase consists of n Pedersen commitments to blocks of length $3k$, followed by a Merkle root over the resulting leaves. After the Freivalds challenge r is sampled, the prover computes the intermediate witnesses $\mathbf{x}=\mathbf{rA}, \mathbf{y}=\mathbf{xB}, \mathbf{z}=\mathbf{rC}$, $\mathbf{b}=\mathbf{sB}$ which costs $\mathcal{O}(k^2)$ field operations. This is distinct from the excluded native computation of $C=AB$. The prover then encodes $\mathbf{x},\mathbf{y},\mathbf{z}$, commits to their encoded symbols, opens the sampled Merkle paths, proves the online CP-SNARK relation, and produces the CP-Link proofs.

For each queried position $j\in I$, the CP-SNARK circuit checks four length-k fold equations:

$$\mathsf{Fold}(\mathbf{r},\widehat{\mathbf{A}}[*,j]), \mathsf{Fold}(\mathbf{x},\widehat{\mathbf{B}}[*,j]),$$

$$\mathsf{Fold}(\mathbf{r},\widehat{\mathbf{C}}[*,j]),\ \mathsf{Fold}(\mathbf{b},\widehat{\mathbf{B}}[*,j])\ .$$

Thus the sampled arithmetic checks contribute $\mathcal{O}(tk)$ constraints.

The intermediate-vector code-consistency checks add E_{code} constraints. For the fixed-rate Reed–Solomon setting used in the experiments, $n=2k$, so this extra cost is linear in k and the total online relation remains linear in k for fixed t.

Verifier work. The verifier does not encode matrices, compute intermediate vectors, or build commitment roots. Its work consists of CP-SNARK verification, verification of the sampled Merkle authentication paths, and CP-Link verification. Therefore, using the notation above, the verifier cost is $E^{\mathcal{V}}_{\mathsf{snark}}+\mathcal{O}(t\log n)\mathbb{H}+E^{\mathcal{V}}_{\mathsf{link}}(k,t)$, where the QALink verifier contributes $\mathcal{O}(t)$ pairing terms.

Comparison. A naive SNARK arithmetization of matrix multiplication costs $\mathcal{O}(k^3)$ constraints, while a Freivalds-based SNARK baseline computes the vector-matrix products inside the circuit and costs $\mathcal{O}(k^2)$ constraints. LAMP moves the vector-matrix products to native prover work and checks sampled encoded-column relations plus intermediate code-consistency equations inside the CP-SNARK. As a result, the main circuit constraint count is $\mathcal{O}(tk+E_{\mathsf{code}})$, which is linear in k for the fixed-rate code used in our implementation.

6. Security Analysis

We prove perfect completeness and computational soundness for the public-coin interactive LAMP scheme. The proof is stated for the protocol in which the verifier samples the Freivalds challenge, the query positions, and the code-check points.

Statement and scope. Section 5 defines the checked relation as the composite relation

$$\mathcal{R}_{\mathsf{LAMP}}=\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}\wedge\mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}.$$

The online component $\mathcal{R}^{\mathsf{on}}_{\mathsf{LAMP}}$ is enforced by the CP-SNARK, while the offline binding component $\mathcal{R}^{\mathsf{off}}_{\mathsf{LAMP}}$ is enforced by sampled Merkle membership checks and CP-Link proofs. This section proves that satisfying both components implies soundness for the committed matrix-multiplication statement.

The input root rt_{ABC}, fixed before the structured challenges, commits to three interleaved words

$$\widehat{\mathbf{A}},\widehat{\mathbf{B}},\widehat{\mathbf{C}}\in\mathbb{F}^{k\times n};$$

these words need not be codewords. Let $0<\tau\le\tau_{\mathsf{UD}}$, where

$$\tau_{\mathsf{UD}}=\frac{1}{n}\left\lfloor\frac{d_{\min}-1}{2}\right\rfloor$$

is the unique-decoding radius of the $[n,k]$ Reed–Solomon code. We call the committed instance false if some input word is more than τ-far from $\mathcal{C}^k$, or if the three words uniquely decode to matrices $\mathbf{A}^*,\mathbf{B}^*,\mathbf{C}^*$ for which $\mathbf{A}^*\mathbf{B}^*\neq\mathbf{C}^*$.

After rt_{ABC} is fixed, the verifier samples independent $r,s\leftarrow\$\ \mathbb{F}$ and sets

$$\mathbf{r}=(1,r,\dots,r^{k-1}),\qquad \mathbf{s}=(1,s,\dots,s^{k-1}).$$

Before the query tuple $I=(j_1,\dots,j_t)\leftarrow\$\ [n]^t$ is sampled, the prover commits to $\mathbf{x},\mathbf{y},\mathbf{z},\mathbf{b}$ and their encoded views. For each $j\in I$, the circuit checks

$$\widehat{\mathbf{x}}[j]=\mathsf{Fold}(\mathbf{r},\widehat{\mathbf{A}}[*,j]),\quad \widehat{\mathbf{y}}[j]=\mathsf{Fold}(\mathbf{x},\widehat{\mathbf{B}}[*,j]),$$
$$\widehat{\mathbf{z}}[j]=\mathsf{Fold}(\mathbf{r},\widehat{\mathbf{C}}[*,j]),\quad \widehat{\mathbf{b}}[j]=\mathsf{Fold}(\mathbf{s},\widehat{\mathbf{B}}[*,j])$$

together with the full-vector equality $\mathbf{y}=\mathbf{z}$. In particular, the relation $\mathbf{b}=\mathbf{sB}$ is one of the four sampled relations; the equality $\mathbf{y}=\mathbf{z}$ is checked in full and incurs no sampling error.

Definition 6.1 (LAMP backend assumptions)**.** Let $\epsilon_{\mathsf{cp}}(\lambda)$, $\epsilon_{\mathsf{open}}(\lambda)$, and $\epsilon_{\mathsf{link}}(\lambda)$ denote, respectively, the soundness errors of the CP-SNARK, the Merkle and Pedersen openings, and CP-Link. Let $\epsilon_{\mathsf{code}}(\lambda)$ be the aggregate error of

LAMP

$\mathcal{P}(\mathbf{A},\mathbf{B},\mathbf{C})$		$\mathcal{V}$
$\widehat{\mathbf{A}},\widehat{\mathbf{B}},\widehat{\mathbf{C}} \leftarrow \mathsf{Enc}(\mathbf{A}),\mathsf{Enc}(\mathbf{B}),\mathsf{Enc}(\mathbf{C}) \in \mathbb{F}^{k\times n}$		
for each $j \in [n]$:		
$\quad \mathbf{m}_{ABC,j} := \big(\widehat{\mathbf{A}}[*,j] \,\\|\, \widehat{\mathbf{B}}[*,j] \,\\|\, \widehat{\mathbf{C}}[*,j]\big) \in \mathbb{F}^{3k}$		
$\quad o_{ABC,j} \leftarrow\\!\\!\\$\ \mathbb{F}$		
$\quad \mathsf{cm}_{ABC,j} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{ABC}, \mathbf{m}_{ABC,j}; o_{ABC,j})$		
$\mathsf{rt}_{ABC} \leftarrow \mathsf{MT.Com}((\mathsf{cm}_{ABC,j})_{j\in[n]}; \perp)$	$\xrightarrow{\mathsf{rt}_{ABC}}$	
	$\xleftarrow{r,\, s}$	$r, s \leftarrow\\!\\!\\$\ \mathbb{F}$
$\mathbf{r} := (1, r, \ldots, r^{k-1})$, $\mathbf{s} := (1, s, \ldots, s^{k-1})$		
$\mathbf{x} \leftarrow \mathbf{rA}, \quad \mathbf{y} \leftarrow \mathbf{xB}, \quad \mathbf{z} \leftarrow \mathbf{rC}, \quad \mathbf{b} \leftarrow \mathbf{sB}$		
$\widehat{\mathbf{x}},\widehat{\mathbf{y}},\widehat{\mathbf{z}},\widehat{\mathbf{b}} \leftarrow \mathsf{Enc}(\mathbf{x}),\mathsf{Enc}(\mathbf{y}),\mathsf{Enc}(\mathbf{z}),\mathsf{Enc}(\mathbf{b})$		
for each $j \in [n]$:		
$\quad \mathbf{m}_{XYZ,j} := \big(\widehat{\mathbf{x}}[j] \,\\|\, \widehat{\mathbf{y}}[j] \,\\|\, \widehat{\mathbf{z}}[j], \,\\|\, \widehat{\mathbf{b}}[j]\big) \in \mathbb{F}^{4n}$		
$\quad o_{XYZ,j} \leftarrow\\!\\!\\$\ \mathbb{F}$		
$\quad \mathsf{cm}_{XYZ,j} \leftarrow \mathsf{Ped.Com}(\mathsf{ck}_{XYZ}, \mathbf{m}_{XYZ,j}; o_{XYZ,j})$		
$\mathsf{rt}_{XYZ} \leftarrow \mathsf{MT.Com}((\mathsf{cm}_{XYZ,j})_{j\in[n]}; \perp)$	$\xrightarrow{\mathsf{rt}_{XYZ}}$	
		$I = (j_1, \ldots, j_t) \leftarrow\\!\\!\\$\ [n]^t$
	$\xleftarrow{I, \alpha_x, \alpha_y, \alpha_z, \alpha_b}$	$\alpha_x, \alpha_y, \alpha_z, \alpha_b \leftarrow\\!\\!\\$\ \mathbb{F} \setminus D$
$\mathcal{T} := (ABC, XYZ)$		
For every $T \in \mathcal{T}$ and every $j \in I$,		
$\quad \pi^T_{\mathsf{mem},j} \leftarrow \mathsf{MT.Open}((\mathsf{cm}_{T,\ell})_{\ell\in[n]}, j)$		
$\mathsf{x} := \big(\mathsf{rt}_{ABC}, \mathsf{rt}_{XYZ}, r, I, \alpha_x, \alpha_y, \alpha_z, \alpha_b\big)$		
$\mathsf{u} := \big((\mathbf{m}_{T,j})_{j\in I}\big)_{T\in\mathcal{T}}, \quad \omega := \big(\mathbf{x},\mathbf{y},\mathbf{z},\mathbf{b},\widehat{\mathbf{x}},\widehat{\mathbf{y}},\widehat{\mathbf{z}},\widehat{\mathbf{b}}\big)$		
$\mathsf{w} := (\mathsf{u}, \omega)$		
For every $T \in \mathcal{T}$,		
$\quad \widetilde{\mathsf{cm}}_T \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Com}(\mathsf{pp}, (\mathbf{m}_{T,j})_{j\in I}; \widetilde{o}_T)$		
$\mathsf{x}_{\mathsf{cp}} \leftarrow \big(\mathsf{x}, (\widetilde{\mathsf{cm}}_T)_{T\in\mathcal{T}}\big)$		
$\pi_{\mathsf{cp}} \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pp}, \mathsf{x}_{\mathsf{cp}}, \mathsf{w})$		
For every $T \in \mathcal{T}$,		
$\quad \pi^T_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^T_{\mathsf{link}}, \mathsf{w}^T_{\mathsf{link}})$		
$\pi_{\mathsf{LAMP}} := \big(\pi_{\mathsf{cp}}, (\widetilde{\mathsf{cm}}_T)_{T\in\mathcal{T}}, \{(\mathsf{cm}_{T,j}, \pi^T_{\mathsf{mem},j})\}_{T\in\mathcal{T}, j\in I}, (\pi^T_{\mathsf{link}})_{T\in\mathcal{T}}\big)$	$\xrightarrow{\pi_{\mathsf{LAMP}}}$	Parse π_{LAMP}
		$\mathsf{x}_{\mathsf{cp}} \leftarrow \big(\mathsf{x}, (\widetilde{\mathsf{cm}}_T)_{T\in\mathcal{T}}\big)$
		$b_{\mathsf{cp}} \leftarrow \mathbf{\Pi}_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}_{\mathsf{cp}}, \pi_{\mathsf{cp}})$
		For every $T \in \mathcal{T}$:
		$\quad$ For every $j \in I$:
		$\qquad b^T_{\mathsf{mem},j} \leftarrow \mathsf{MT.Verify}(\mathsf{rt}_T, j, \mathsf{cm}_{T,j}, \pi^T_{\mathsf{mem},j})$
		$\quad b^T_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^T_{\mathsf{link}}, \pi^T_{\mathsf{link}})$
		Accept iff $b_{\mathsf{cp}} \wedge \bigwedge_{T\in\mathcal{T}} \Big(b^T_{\mathsf{link}} \wedge \bigwedge_{j\in I} b^T_{\mathsf{mem},j}\Big)$

Figure 3. The LAMP protocol

the encoding checks for $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}, \widehat{\mathbf{b}}$ sampled after the intermediate commitment is fixed. For the four independent barycentric Reed–Solomon checks used in the protocol,

$$\epsilon_{\mathsf{code}} \le \frac{4(n-1)}{|\mathbb{F}| - n}.$$

Theorem 6.2 (Completeness, soundness, and zero knowledge)**.** *Let $\mathcal{C}$ be the $[n, k]$ Reed–Solomon code with relative distance $\delta_{\mathsf{RS}} = (n - k + 1)/n$, and assume Definition 6.1. Then the public-coin interactive* LAMP *scheme satisfies:*

(i) ***Perfect completeness.*** *Honest encodings of matrices satisfying $\mathbf{AB} = \mathbf{C}$ are accepted with probability one.*

(ii) ***Interactive computational soundness.*** *For every* PPT *prover and every false committed instance fixed before r, s,*

$$\begin{aligned}\Pr[\mathsf{Acc}] \le\ & \epsilon_{\mathsf{cp}}(\lambda) + \epsilon_{\mathsf{open}}(\lambda) + \epsilon_{\mathsf{link}}(\lambda) + \epsilon_{\mathsf{code}}(\lambda) \\ & + \frac{(k-1)n}{|\mathbb{F}|} + \frac{k-1}{|\mathbb{F}|} \\ & + \max\bigl\{(1-\tau)^t, (1-\delta_{\mathsf{RS}}+\tau)^t\bigr\}.\end{aligned}$$

(iii) ***Zero knowledge.*** *If the CP-SNARK is zero knowledge, CP-Link is witness zero knowledge, and Pedersen commitments are hiding, the verifier's view is computationally simulatable from the public transcript.*

Proof sketch. Set

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}, \qquad \mathbf{b} = \mathbf{sB}.$$

When $\mathbf{AB} = \mathbf{C}$, we have $\mathbf{y} = \mathbf{z}$; the four fold equations follow from the linearity of the code. This proves completeness.

For soundness, condition on the correctness of the four backend checks. If an input word is τ-far from $\mathcal{C}^k$, choose the first such word in the order $\widehat{\mathbf{A}}, \widehat{\mathbf{B}}, \widehat{\mathbf{C}}$. This choice is determined by rt_{ABC} and hence precedes r, s. Apply the structured-fold proximity bound to its $\mathbf{r}$-fold when the chosen word is $\widehat{\mathbf{A}}$ or $\widehat{\mathbf{C}}$, and to its $\mathbf{s}$-fold when it is $\widehat{\mathbf{B}}$. The exceptional probability is at most $(k-1)n/|\mathbb{F}|$. Otherwise the chosen fold is more than τ-far from the claimed intermediate codeword, and all t queries avoid the disagreement set with probability at most $(1-\tau)^t$.

It remains to consider the case in which all three words uniquely decode to $\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*$. If $\mathbf{A}^*\mathbf{B}^* \ne \mathbf{C}^*$, Schwartz–Zippel gives

$$\Pr_r[\mathbf{r}(\mathbf{A}^*\mathbf{B}^* - \mathbf{C}^*) = 0] \le \frac{k-1}{|\mathbb{F}|}.$$

Except for this event, $\mathbf{y} = \mathbf{z}$ is incompatible with all four fold relations. This includes $\mathbf{b} = \mathbf{sB}^*$, which tests the middle input independently of $\mathbf{x}$. Choose a false fold relation, in a fixed order, after the intermediate commitment and before I is sampled. Its correct folded word is τ-close to one codeword, whereas the prover's claimed word is a distinct codeword. The two tested words therefore differ in at least $(\delta_{\mathsf{RS}} - \tau)n$ positions, giving failure probability at most $(1-\delta_{\mathsf{RS}}+\tau)^t$. Since the relation is fixed before the queries, no union bound over the four relations is needed. The equality $\mathbf{y} = \mathbf{z}$ is a full-vector check rather than an additional sampled relation.

Zero knowledge follows from the simulators for the CP-SNARK and CP-Link and the hiding of the Pedersen commitments. Appendix C gives the full argument. □

Remark 3 (Fiat–Shamir). The theorem above analyzes the public-coin interactive protocol. A non-interactive variant can derive r, I, and the code-check points from domain-separated transcript hashes via the Fiat–Shamir transform. Proving that variant adds the standard random oracle model loss and is orthogonal to the interactive soundness statement above.

7. Implementation and Evaluation

We implement LAMP and evaluate its performance. The implementation follows the construction in Section 5. We instantiate the CP-SNARK with Groth16, using the commit-and-prove interface [10], [33]. For CP-Link, we use a pairing-based QA-NIZK proof [37]; the concrete instantiation used in our implementation is given in Appendix E. The implementation is available at https://anonymous.4open.science/r/LAMP-38E3/.

Our evaluation is organized around four questions. First, does the number of constraints of LAMP scale linearly with the matrix dimension, as predicted by Section 5.5? Second, which components dominate proving time, proof size, and verification time? Third, how does LAMP behave when proving multiple independent claims $\mathbf{A}_i\mathbf{B}_i = \mathbf{C}_i$ in a batch, rather than a single product $\mathbf{AB} = \mathbf{C}$? Fourth, does the advantage remain useful on a neural-network workload?

Experimental setup. We implement both LAMP and the Freivalds baseline in Go using gnark, with BN254 as the elliptic curve. All experiments were run on a Linux server with an AMD EPYC 7B13 CPU with 32 cores and 256GB of memory. Each experiment was repeated ten times, and we report the average over these runs. We use a Reed–Solomon code of rate $\rho = 1/2$ and set the number of sampled positions to $t = 309$.

7.1. Comparison with Matrix-Multiplication Proof Systems

We compare LAMP with existing proof systems for matrix multiplication. Although zkMatrix and zkMaP are included in the theoretical comparison in Table 2, we were unable to identify publicly available implementations that would allow us to reproduce their experiments. We therefore compare LAMP against two systems: a Freivalds-based baseline implemented as an arithmetic circuit using LegoSNARK, and DualMatrix, for which

k	System	Constraints	Prove Time	Verify Time	Proof Size
2^7	LAMP	–	–	–	–
	Freivalds	49,610	0.59 s	2 ms	196 B
	DualMatrix	–	3.57 s	0.13 s	71,288 B
2^8	LAMP	329,377	2.94 s	0.13 s	50,308 B
	Freivalds	197,194	2.02 s	2 ms	196 B
	DualMatrix	–	4.81 s	0.14 s	77,140 B
2^9	LAMP	655,245	5.73 s	0.14 s	65,860 B
	Freivalds	788,810	6.99 s	2 ms	196 B
	DualMatrix	–	7.98 s	0.15 s	82,992 B
2^{10}	LAMP	1,306,972	12.56 s	0.13 s	80,772 B
	Freivalds	3,156,298	25.91 s	1 ms	196 B
	DualMatrix	–	18.54 s	0.15 s	88,844 B
2^{11}	LAMP	2,609,193	27.49 s	0.13 s	99,012 B
	Freivalds	12,622,154	102.32 s	2 ms	196 B
	DualMatrix	–	57.02 s	0.16 s	94,696 B
2^{12}	LAMP	5,214,877	69.95 s	0.13 s	119,556 B
	Freivalds	50,495,818	411.28 s	2 ms	196 B
	DualMatrix	–	205.21 s	0.17 s	100,548 B
2^{13}	LAMP	10,426,236	193.89 s	0.13 s	136,260 B
	Freivalds	–	–	–	–
	DualMatrix	–	707.43 s	0.18 s	106,400 B

TABLE 3. Performance comparison of LAMP, the Freivalds baseline, and DualMatrix for square matrix multiplication.

a public implementation is available. Table 3 reports the constraint count, proving time, verification time, and proof size of these systems. For both LAMP and DualMatrix, proving time includes commitment generation, and proof size includes the corresponding commitment data.

For the LAMP experiments, we use a code rate of $\rho = 1/2$, which requires $t = 309$ queried positions. Since the codeword length is $n = 2k$, at $k = 2^7$ we have $n = 256 < t$. Consequently, the LAMP experiments begin at $k = 2^8$. DualMatrix, on the other hand, is a specialized matrix-multiplication proof system that does not use an arithmetic circuit. An R1CS constraint count is therefore not an applicable metric for DualMatrix, and its constraint entries are left unspecified in the table.

For the Freivalds baseline, the number of constraints increases by approximately $4\times$ whenever the matrix dimension k doubles. This is because the vector–matrix products performed inside the circuit require $\mathcal{O}(k^2)$ constraints. Moreover, its proving complexity is approximately $\mathcal{O}(k^2 \log k)$, causing the proving-time gap between the baseline and LAMP to grow with k. At $k = 2^8$, the Freivalds baseline is faster, requiring 2.02 seconds compared with 2.94 seconds for LAMP. Starting at $k = 2^9$, however, LAMP becomes faster. At $k = 2^{12}$, LAMP reduces the constraint count from 50,495,818 to 5,214,877, a $9.68\times$ reduction, and reduces proving time from 411.28 seconds to 69.95 seconds, corresponding to a $5.88\times$ speedup.

The Freivalds baseline cannot complete the experiment at $k = 2^{13}$ because of insufficient memory. In contrast, LAMP completes the same experiment using approximately 50.57 GB of memory, with a proving time of 193.89 seconds. Because the Freivalds baseline uses the constant-size Groth16 proof provided by LegoSNARK, its proof size remains 196 B and its verification time remains approximately 1–2 ms, independently of the constraint count. Thus, the Freivalds baseline provides very small proofs and fast verification, but its proving time and memory consumption increase rapidly for large matrices.

Across the range shared by LAMP and DualMatrix, $2^8 \leq k \leq 2^{13}$, LAMP achieves a shorter proving time at every measured matrix dimension. At $k = 2^8$, LAMP requires 2.94 seconds, whereas DualMatrix requires 4.81 seconds, making LAMP approximately $1.64\times$ faster. The gap increases with the matrix dimension: at $k = 2^{13}$, the respective proving times are 193.89 seconds and 707.43 seconds, giving LAMP a $3.65\times$ speedup. Verification times are comparable and remain below 0.2 seconds for both systems, although LAMP is consistently slightly faster across all shared measurements.

Proof size exhibits the opposite trend. From $k = 2^8$ through $k = 2^{10}$, LAMP produces smaller proofs than DualMatrix. Starting at $k = 2^{11}$, however, the proofs generated by LAMP become larger. For example, at $k = 2^{13}$, the proof sizes are 136,260 B for LAMP and 106,400 B for DualMatrix. These results demonstrate that LAMP provides faster proof generation and slightly faster verification than DualMatrix, at the cost of larger proofs for sufficiently large matrices.

k	Setup Time		Commit Time			Prove Time			Verify Time			Proof Size		
	Groth16	CP-Link	Encode	ABC	XYZ	Merkle	Groth16	CP-Link	Groth16	Merkle	CP-Link	Merkle	Groth16	CP-Link
2^8	20.03 s	1.28 s	0.04 s	0.21 s	0.02 s	1 ms	2.59 s	0.08 s	2 ms	1 ms	0.13 s	49,920 B	260 B	128 B
2^9	39.96 s	2.52 s	0.14 s	0.74 s	0.05 s	1 ms	4.68 s	0.12 s	2 ms	2 ms	0.14 s	65,472 B	260 B	128 B
2^{10}	80.58 s	5.01 s	0.46 s	2.66 s	0.14 s	1 ms	9.09 s	0.21 s	2 ms	2 ms	0.13 s	80,384 B	260 B	128 B
2^{11}	158.39 s	9.94 s	1.51 s	7.83 s	0.54 s	1 ms	17.16 s	0.43 s	2 ms	3 ms	0.13 s	98,624 B	260 B	128 B
2^{12}	316.07 s	19.61 s	5.13 s	27.17 s	2.68 s	1 ms	34.15 s	0.81 s	2 ms	3 ms	0.13 s	119,168 B	260 B	128 B
2^{13}	637.66 s	39.42 s	18.09 s	99.00 s	8.37 s	1 ms	66.95 s	1.49 s	2 ms	4 ms	0.13 s	135,872 B	260 B	128 B

TABLE 4. COMPONENT-LEVEL MEASUREMENTS FOR LAMP.

7.2. LAMP Component Breakdown

Table 4 presents a component-wise breakdown of the costs of LAMP with the code rate set to $\rho = 1/2$. For small matrices, Groth16-based proof generation is the dominant cost. As k increases, however, encoding and commitment generation account for a larger fraction of the total cost. At $k = 2^8$, encoding and commitment generation take 0.04 s and 0.23 s, respectively, whereas generating the Merkle, Groth16, and CP-Link proofs takes 2.67 s in total. In contrast, at $k = 2^{13}$, encoding and commitment generation take 125.46 s in total, exceeding the proof-generation cost of 68.44 s. This result shows that the primary bottleneck shifts from proof generation to encoding and commitment generation as the matrix dimension increases.

LAMP requires trusted setup for both Groth16 and CP-Link, with the setup cost dominated by Groth16. The Groth16 setup time increases from 20.03 s at $k = 2^8$ to 637.66 s at $k = 2^{13}$, whereas the CP-Link setup time increases from 1.28 s to 39.42 s over the same range.

Verification time remains nearly constant as the matrix dimension increases. Groth16 and Merkle verification each take only a few milliseconds, while CP-Link verification takes approximately 0.13–0.14 s. The Groth16 and CP-Link proof sizes also remain constant at 260 B and 128 B, respectively. Consequently, the total proof size is dominated by the Merkle proof, which increases from $49{,}920$ B to $135{,}872$ B over the measured range.

Table 7 in Appendix F presents additional results for $\rho \in \{1/2, 1/4, 1/8\}$. A higher code rate reduces encoding and commitment costs by shortening the codeword, but it also decreases the relative distance and therefore requires more queries. Specifically, the required numbers of queries for $\rho = 1/8$, $1/4$, and $1/2$ are $t = 155$, 189, and 309, respectively. Thus, the code rate determines the trade-off between encoding and commitment costs and the generation time and size of the sampled proof.

7.3. Batching Matrix-Multiplication Claims

We evaluate batching multiple independent claims $\mathbf{A}_i\mathbf{B}_i = \mathbf{C}_i$. At each codeword position, the prover packs the symbols of all claims into a single Pedersen leaf and constructs one shared Merkle tree. Consequently, the number of Merkle membership proofs is independent of the batch size.

Table 5 reports the component costs at $k = 2^8$ and $\rho = 1/2$. As the number of claims increases, the constraint count and Groth16 proving time grow, while the Merkle proof size remains approximately 50 KB and verification time remains approximately 0.13 s. This is because the sampled positions and the shared Merkle openings are independent of the number of claims; minor size variations result from shared authentication nodes in the multi-opening implementation.

7.4. GPT-2 Workload

Table 6 reports the results for a GPT-2 layer with sequence length 2^{10} and 36 matrix-multiplication claims. To accommodate the dimensions of the matrices in the layer, LAMP uses $\rho = 1/4$ and $t = 189$. As in Section 7.1, proving times include encoding and commitment generation, and proof sizes include commitment data. For DualMatrix, we configure the inputs of its public implementation to match the GPT-2 matrix structures and evaluate the same matrix multiplications.

Compared with the Freivalds baseline, LAMP reduces the constraint count from $76{,}807{,}671$ to $46{,}730{,}248$ and the proving time from 408.69 s to 367.40 s. DualMatrix requires 519.97 s, making LAMP the fastest prover among the three systems. In contrast, the Freivalds baseline provides the fastest verification and smallest proof, while DualMatrix also verifies faster and produces a smaller proof than LAMP.

The relatively large proof size of LAMP in the GPT-2 experiment stems from the different dimensions of the matrices involved in the workload. These matrix multiplications require multiple commitment layouts and corresponding CP-Link proofs and Merkle membership proofs. These linking and membership proofs—dominated in size by the Merkle openings—therefore account for a substantial portion of the total proof and are the main source of proof-size growth in heterogeneous workloads such as GPT-2.

Claims	Constraints	Commit Time			Prove Time			Proof Size		
		Encode	ABC	XYZ	Merkle	Groth16	CP-Link	Merkle	Groth16	CP-Link
1	329,377	0.11 s	0.22 s	0.02 s	0 ms	2.62 s	0.07 s	49,728 B		
2	649,929	0.11 s	0.38 s	0.02 s	0 ms	4.81 s	0.12 s	50,240 B		
3	970,481	0.12 s	0.54 s	0.03 s	0 ms	6.49 s	0.15 s	49,088 B		
4	1,291,033	0.12 s	0.69 s	0.04 s	0 ms	8.71 s	0.18 s	49,984 B		
5	1,611,585	0.13 s	0.83 s	0.05 s	0 ms	10.49 s	0.22 s	50,304 B	260 B	128 B
6	1,932,137	0.13 s	0.80 s	0.05 s	0 ms	12.43 s	0.26 s	49,600 B		
7	2,252,689	0.13 s	0.94 s	0.05 s	0 ms	14.92 s	0.30 s	49,600 B		
8	2,573,241	0.14 s	1.02 s	0.06 s	0 ms	16.97 s	0.45 s	49,792 B		
9	2,893,793	0.14 s	1.19 s	0.06 s	0 ms	18.83 s	0.47 s	49,600 B		
10	3,214,345	0.15 s	1.25 s	0.07 s	0 ms	20.61 s	0.56 s	49,856 B		

TABLE 5. BATCHING q INDEPENDENT MATRIX-MULTIPLICATION CLAIMS $\{\mathbf{A}_i\mathbf{B}_i = \mathbf{C}_i\}_{i\in[q]}$ AT FIXED $k = 2^8$.

System	Constraints	Prove	Verify	Proof Size
LAMP	46,730,248	367.40 s	10.53 s	6,298,468 B
Freivalds	76,807,671	408.69 s	0.49 s	196 B
DualMatrix	–	519.97 s	5.22 s	2,934,952 B

TABLE 6. GPT-2 AT SEQUENCE LENGTH 2^{10} WITH 36 MATRIX-MULTIPLICATION CLAIMS.

8. Conclusion

We proposed LAMP, a protocol for verifying matrix multiplication by moving expensive vector–matrix products outside the SNARK circuit and checking them through sampled local tests over encoded data. The prover commits to the encoded matrices and Freivalds intermediate vectors, while the SNARK circuit proves only the fold relations at the sampled positions. Merkle openings and CP-Link proofs ensure that the values used inside the CP-SNARK are consistent with the externally committed data fixed before the sampling challenges are known.

Our implementation confirms the intended tradeoff. For a fixed number t of queries, the main arithmetic relation has $\mathcal{O}(tk + E_{\mathsf{code}})$ constraints rather than the $\mathcal{O}(k^2)$ constraints of a Freivalds baseline. Thus, when t is fixed and the code-consistency checks are linear, the constraint count of LAMP grows linearly with the matrix dimension k.

The experimental results show that this structural advantage becomes more pronounced as the matrix dimension increases. At $k = 2^{12}$, LAMP reduces the constraint count of the Freivalds baseline from $50{,}495{,}818$ to $5{,}214{,}877$, a $9.68\times$ reduction, and decreases the proving time from 411.28 s to 69.95 s, a $5.88\times$ speedup. LAMP also achieves faster proof generation than DualMatrix at every matrix dimension measured for both systems, with a $3.65\times$ speedup at $k = 2^{13}$.

When multiple matrix-multiplication claims share the same leaf structure, batching amortizes the cost of Merkle openings, and the Merkle proof size and verification time remain nearly constant as the number of matrix multiplications increases. On the GPT-2 workload, LAMP also achieves faster proof generation than both the Freivalds baseline and DualMatrix. However, jointly processing matrix multiplications of different dimensions requires multiple commitment layouts and the corresponding CP-Link proofs and Merkle membership proofs. Consequently, heterogeneous workloads such as GPT-2 incur larger proof sizes and higher verification costs.

As future work, we first plan to apply the Fiat–Shamir transform to LAMP, which is currently specified as an interactive protocol, and establish a rigorous security proof for the resulting non-interactive protocol. We also plan to reduce proof size by efficiently sharing commitment layouts and opening information across consecutive matrix multiplications of different dimensions, enabling more efficient deployment in verifiable AI workloads. We expect these improvements to further reduce proof size and verification cost while preserving the linear constraint complexity of LAMP.

References

[1] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*. Berlin, Heidelberg: Springer, 2016, pp. 305–326.

[2] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953

[3] M. Maller, S. Bowe, M. Kohlweiss, and S. Meiklejohn, "Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2019, pp. 2111–2128.

[4] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, "Marlin: Preprocessing zkSNARKs with universal and updatable SRS," in *Advances in Cryptology – EUROCRYPT 2020*. Cham: Springer, 2020, pp. 738–768.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[8] R. Freivalds, "Probabilistic machines can use less running time," in *Information Processing 77 (Proceedings of IFIP Congress 77)*. Amsterdam, The Netherlands: North-Holland, 1977, pp. 839–842.

[9] B. Parno, C. Gentry, J. Howell, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," Cryptology ePrint Archive, Paper 2013/279, 2013. [Online]. Available: https://eprint.iacr.org/2013/279

[10] M. Campanelli, D. Fiore, and A. Querol, "LegoSNARK: Modular design and composition of succinct zero-knowledge proofs," Cryptology ePrint Archive, Paper 2019/142, 2019. [Online]. Available: https://eprint.iacr.org/2019/142

[11] J. Thaler, "Time-optimal interactive proofs for circuit evaluation," in *Advances in Cryptology – CRYPTO 2013*. Berlin, Heidelberg: Springer, 2013, pp. 71–89.

[12] M. Cong, T. H. Yuen, and S.-M. Yiu, "zkMatrix: Batched short proof for committed matrix multiplication," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2024, pp. 289–305.

[13] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell, "Bulletproofs: Short proofs for confidential transactions and more," in *2018 IEEE Symposium on Security and Privacy*. New York, NY, USA: IEEE, 2018, pp. 315–334.

[14] M. Cong, T. H. Yuen, and S. M. Yiu, "DualMatrix: Conquering zkSNARK for large matrix multiplication," *Cybersecurity*, vol. 9, no. 1, p. 126, 2026.

[15] B. Deressa and M. A. Hasan, "zkmap: Zero-knowledge succinct non-interactive matrix multiplication proofs," *IACR Communications in Cryptology*, vol. 2, no. 3, 2025.

[16] S. Lee, H. Ko, J. Kim, and H. Oh, "vCNN: Verifiable convolutional neural network based on zk-SNARKs," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 4254–4270, 2024.

[17] J. Weng, J. Weng, G. Tang, A. Yang, M. Li, and J.-N. Liu, "pvcnn: Privacy-preserving and verifiable convolutional neural network testing," 2023. [Online]. Available: https://arxiv.org/abs/2201.09186

[18] Y. Zhang, M. Zheng, X. Chen, J. Hu, W. Shi, L. Ju, Y. Solihin, and Q. Lou, "zkVC: Fast zero-knowledge proof for private and verifiable computing," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE. New York, NY, USA: IEEE, 2025, pp. 1–7.

[19] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2021, pp. 2968–2985.

[20] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, "zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference," Cryptology ePrint Archive, Paper 2025/1184, 2025. [Online]. Available: https://eprint.iacr.org/2025/1184

[21] H. Sun, J. Li, and H. Zhang, "zkllm: Zero knowledge proofs for large language models," 2024. [Online]. Available: https://arxiv.org/abs/2404.16109

[22] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, "Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning," in *30th USENIX Security Symposium (USENIX Security 21)*. Berkeley, CA, USA: USENIX Association, 2021, pp. 501–518.

[23] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "zkML: An optimizing system for ML inference in zero-knowledge proofs," in *Proceedings of the Nineteenth European Conference on Computer Systems*. New York, NY, USA: ACM, 2024, pp. 560–574.

[24] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Fast Reed-Solomon Interactive Oracle Proofs of Proximity," in *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), I. Chatzigiannakis, C. Kaklamanis, D. Marx, and D. Sannella, Eds., vol. 107. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, pp. 14:1–14:17. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2018.14

[25] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev, "STIR: Reed–solomon proximity testing with fewer queries," Cryptology ePrint Archive, Paper 2024/390, 2024. [Online]. Available: https://eprint.iacr.org/2024/390

[26] ——, "WHIR: Reed–solomon proximity testing with super-fast verification," Cryptology ePrint Archive, Paper 2024/1586, 2024. [Online]. Available: https://eprint.iacr.org/2024/1586

[27] H. Zeilberger, B. Chen, and B. Fisch, "BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes," Cryptology ePrint Archive, Paper 2023/1705, 2023. [Online]. Available: https://eprint.iacr.org/2023/1705

[28] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam, "Ligero: Lightweight sublinear arguments without a trusted setup," Cryptology ePrint Archive, Paper 2022/1608, 2022. [Online]. Available: https://eprint.iacr.org/2022/1608

[29] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby, "Brakedown: Linear-time and field-agnostic snarks for r1cs," in *Advances in Cryptology – CRYPTO 2023*. Cham: Springer, 2023, pp. 193–226.

[30] M. Brehm, B. Chen, B. Fisch, N. Resch, R. D. Rothblum, and H. Zeilberger, "Blaze: Fast SNARKs from interleaved RAA codes," Cryptology ePrint Archive, Paper 2024/1609, 2024. [Online]. Available: https://eprint.iacr.org/2024/1609

[31] T. Xie, Y. Zhang, and D. Song, "Orion: Zero knowledge proof with linear prover time," Cryptology ePrint Archive, Paper 2022/1010, 2022. [Online]. Available: https://eprint.iacr.org/2022/1010

[32] T. den Hollander and D. Slamanig, "A crack in the firmament: Restoring soundness of the orion proof system and more," Cryptology ePrint Archive, Paper 2024/1164, 2024. [Online]. Available: https://eprint.iacr.org/2024/1164

[33] A. Belling, A. Soleimanian, and O. Bégassat, "Recursion over public-coin interactive proof systems; faster hash verification," Cryptology ePrint Archive, Paper 2022/1072, 2022. [Online]. Available: https://eprint.iacr.org/2022/1072

[34] J. T. Schwartz, "Fast probabilistic algorithms for verification of polynomial identities," *Journal of the ACM*, vol. 27, no. 4, pp. 701–717, 1980.

[35] R. Zippel, "Probabilistic algorithms for sparse polynomials," in *Symbolic and Algebraic Computation (EUROSAM '79)*. Berlin, Heidelberg: Springer, 1979, pp. 216–226.

[36] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf, "Proximity gaps for reed-solomon codes," Cryptology ePrint Archive, Paper 2020/654, 2020. [Online]. Available: https://eprint.iacr.org/2020/654

[37] E. Kiltz and H. Wee, "Quasi-adaptive NIZK for linear subspaces revisited," Cryptology ePrint Archive, Paper 2015/216, 2015. [Online]. Available: https://eprint.iacr.org/2015/216

Appendix A. SNARK Security Definitions

Let $\mathcal{R} = \{\mathcal{R}_\lambda\}_{\lambda\in\mathbb{N}}$ be a family of NP relations. A SNARK for $\mathcal{R}$ is a tuple of algorithms $\mathbf{\Pi} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$, as defined in Section 3.4. We recall the standard security properties used in this work.

Completeness. $\mathbf{\Pi}$ satisfies completeness if, for every security parameter λ and every $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_\lambda$,

$$\Pr\left[\begin{array}{l}\mathbf{\Pi}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \pi) = 1\\ \mathsf{pp} \leftarrow \mathbf{\Pi}.\mathsf{Setup}(1^\lambda, \mathcal{R}_\lambda), \quad \pi \leftarrow \mathbf{\Pi}.\mathsf{Prove}(\mathsf{pp}, \mathsf{x}, \mathsf{w})\end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

If the probability is 1, we say that $\mathbf{\Pi}$ has perfect completeness.

Knowledge soundness. $\mathbf{\Pi}$ satisfies knowledge soundness if, for every PPT prover $\mathcal{A}$, there exists a PPT extractor $\mathsf{Ext}^{\mathcal{A}}$ such that

$$\Pr\left[\begin{array}{c}\mathbf{\Pi}.\mathsf{Verify}(\mathsf{pp}, \mathsf{x}, \pi) = 1\\ \wedge\ (\mathsf{x}, \mathsf{w}) \notin \mathcal{R}_\lambda\end{array} : \begin{array}{l}\mathsf{pp} \leftarrow \mathbf{\Pi}.\mathsf{Setup}(1^\lambda, \mathcal{R}_\lambda),\\ (\mathsf{x}, \pi) \leftarrow \mathcal{A}(\mathsf{pp}),\\ \mathsf{w} \leftarrow \mathsf{Ext}^{\mathcal{A}}(\mathsf{pp}, \mathsf{x})\end{array}\right] \leq \mathsf{negl}(\lambda).$$

Equivalently, except with negligible probability, any prover that outputs an accepting proof for x must know a witness w such that $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_\lambda$.

Zero knowledge. $\mathbf{\Pi}$ satisfies zero knowledge if there exists a PPT simulator Sim such that, for every PPT distinguisher $\mathcal{D}$, the following two experiments are indistinguishable.

$$\mathsf{Real}_{\mathbf{\Pi},\mathcal{D}}(\lambda) : \begin{cases}\mathsf{pp} \leftarrow \mathbf{\Pi}.\mathsf{Setup}(1^\lambda, \mathcal{R}_\lambda),\\ \pi \leftarrow \mathbf{\Pi}.\mathsf{Prove}(\mathsf{pp}, \mathsf{x}, \mathsf{w}),\\ b \leftarrow \mathcal{D}(\mathsf{pp}, \mathsf{x}, \pi),\\ \text{return } b.\end{cases}$$

$$\mathsf{Ideal}_{\mathbf{\Pi},\mathcal{D}}(\lambda) : \begin{cases}\mathsf{pp} \leftarrow \mathbf{\Pi}.\mathsf{Setup}(1^\lambda, \mathcal{R}_\lambda),\\ \pi^\star \leftarrow \mathsf{Sim}(\mathsf{pp}, \mathsf{x}),\\ b \leftarrow \mathcal{D}(\mathsf{pp}, \mathsf{x}, \pi^\star),\\ \text{return } b.\end{cases}$$

For all $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_\lambda$,

$$|\Pr[\mathsf{Real}_{\mathbf{\Pi},\mathcal{D}}(\lambda) = 1] - \Pr[\mathsf{Ideal}_{\mathbf{\Pi},\mathcal{D}}(\lambda) = 1]| \leq \mathsf{negl}(\lambda).$$

A SNARK satisfying zero knowledge is called a zk-SNARK.

Appendix B. CP-Link Security Definitions

Let $\mathcal{R}_{\mathsf{link}} = \{\mathcal{R}_{\mathsf{link},\lambda}\}_{\lambda\in\mathbb{N}}$ be the blockwise linking relation defined in Section 3.5. For a SNARK-side key ck_S, an external key ck_E, a SNARK-side commitment $\widetilde{\mathsf{cm}}$, external commitments $\mathsf{cm}_0, \ldots, \mathsf{cm}_{q-1}$, sampled blocks $(\mathbf{m}_i)_{i=0}^{q-1}$, and opening randomness $\widetilde{o}, o_0, \ldots, o_{q-1}$, define

$$\begin{aligned}\mathsf{x}_{\mathsf{link}} &= ((\mathsf{ck}_S, \widetilde{\mathsf{cm}}), (\mathsf{ck}_E, \mathsf{cm}_0, \ldots, \mathsf{cm}_{q-1})),\\ \mathsf{w}_{\mathsf{link}} &= ((\mathbf{m}_i)_{i=0}^{q-1}, \widetilde{o}, (o_i)_{i=0}^{q-1}).\end{aligned}$$

The relation $\mathcal{R}_{\mathsf{link},\lambda}$ contains exactly the pairs $(\mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}})$ satisfying

$$\begin{aligned}\widetilde{\mathsf{cm}} &= \mathsf{Com}(\mathsf{ck}_S, \mathbf{m}_0 \| \cdots \| \mathbf{m}_{q-1}; \widetilde{o}),\\ \mathsf{cm}_i &= \mathsf{Com}(\mathsf{ck}_E, \mathbf{m}_i; o_i) \quad \text{for all } i = 0, \ldots, q-1.\end{aligned}$$

A CP-Link proof system for $\mathcal{R}_{\mathsf{link}}$ is a tuple of algorithms $\mathbf{\Pi}_{\mathsf{link}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$, as defined in Section 3.5. The corresponding linking language is

$$\mathcal{L}_{\mathsf{link},\lambda} = \{\mathsf{x}_{\mathsf{link}} : \exists \mathsf{w}_{\mathsf{link}} \text{ such that } (\mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}}) \in \mathcal{R}_{\mathsf{link},\lambda}\}.$$

Thus a statement is in $\mathcal{L}_{\mathsf{link},\lambda}$ exactly when the SNARK-side commitment opens to the ordered concatenation of sampled blocks and each external commitment opens to the corresponding block. The main security property for our protocol is link soundness. If the CP-Link backend is implemented as a NIZK or QA-NIZK, as in our implementation, it may additionally satisfy witness zero knowledge [10], [37].

Link completeness. $\mathbf{\Pi}_{\mathsf{link}}$ satisfies completeness if, for every security parameter λ and every $(\mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}}) \in \mathcal{R}_{\mathsf{link},\lambda}$,

$$\Pr\left[\begin{array}{l}\mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \pi_{\mathsf{link}}) = 1\\ \mathsf{pp}_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{link},\lambda}),\\ \pi_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}})\end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

If the probability is 1, we say that $\mathbf{\Pi}_{\mathsf{link}}$ has perfect completeness.

Link soundness. $\mathbf{\Pi}_{\mathsf{link}}$ satisfies link soundness with error $\epsilon_{\mathsf{link}}(\lambda)$ if, for every PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{l}\mathbf{\Pi}_{\mathsf{link}}.\mathsf{Verify}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}^\star_{\mathsf{link}}, \pi^\star_{\mathsf{link}}) = 1\\ \ \wedge\ \mathsf{x}^\star_{\mathsf{link}} \notin \mathcal{L}_{\mathsf{link},\lambda}\\ \mathsf{pp}_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{link},\lambda}),\\ (\mathsf{x}^\star_{\mathsf{link}}, \pi^\star_{\mathsf{link}}) \leftarrow \mathcal{A}(\mathsf{pp}_{\mathsf{link}})\end{array}\right] \leq \epsilon_{\mathsf{link}}(\lambda).$$

If $\epsilon_{\mathsf{link}}(\lambda) = \mathsf{negl}(\lambda)$, we say that $\mathbf{\Pi}_{\mathsf{link}}$ is link-sound. Equivalently, except with probability $\epsilon_{\mathsf{link}}(\lambda)$, an accepting

CP-Link proof certifies the existence of blocks $(\mathbf{m}_i)_{i=0}^{q-1}$, an aggregate opening $\widetilde{o}$, and external openings $(o_i)_{i=0}^{q-1}$ such that $\widetilde{\mathsf{cm}}$ opens to $\mathbf{m}_0\|\cdots\|\mathbf{m}_{q-1}$ and each cm_i opens to $\mathbf{m}_i$.

Witness zero knowledge. When privacy of the linked blocks and opening randomness is required, we say that $\mathbf{\Pi}_{\mathsf{link}}$ satisfies witness zero knowledge if there exists a PPT simulator $\mathsf{Sim}_{\mathsf{link}}$ such that, for every PPT distinguisher $\mathcal{D}$, the following two experiments are indistinguishable.

$$\mathsf{Real}_{\mathsf{link},\mathcal{D}}(\lambda):\quad \begin{cases} \mathsf{pp}_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{link},\lambda}),\\ \pi_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Prove}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}}),\\ b \leftarrow \mathcal{D}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \pi_{\mathsf{link}}),\\ \text{return } b. \end{cases}$$

$$\mathsf{Ideal}_{\mathsf{link},\mathcal{D}}(\lambda):\quad \begin{cases} \mathsf{pp}_{\mathsf{link}} \leftarrow \mathbf{\Pi}_{\mathsf{link}}.\mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{link},\lambda}),\\ \pi^\star_{\mathsf{link}} \leftarrow \mathsf{Sim}_{\mathsf{link}}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}),\\ b \leftarrow \mathcal{D}(\mathsf{pp}_{\mathsf{link}}, \mathsf{x}_{\mathsf{link}}, \pi^\star_{\mathsf{link}}),\\ \text{return } b. \end{cases}$$

For all $(\mathsf{x}_{\mathsf{link}}, \mathsf{w}_{\mathsf{link}}) \in \mathcal{R}_{\mathsf{link},\lambda}$,

$$|\Pr[\mathsf{Real}_{\mathsf{link},\mathcal{D}}(\lambda) = 1] - \Pr[\mathsf{Ideal}_{\mathsf{link},\mathcal{D}}(\lambda) = 1]| \le \mathsf{negl}(\lambda).$$

This property says that the linking proof reveals no information about the blocks $(\mathbf{m}_i)_{i=0}^{q-1}$ or the openings $\widetilde{o}, o_0, \ldots, o_{q-1}$, beyond the public fact that the commitments are blockwise linkable.

Appendix C. Detailed Proof of Theorem 6.2

Proof. We consider the public-coin interactive protocol.

Perfect completeness. Let the honest prover hold $\mathbf{A}, \mathbf{B}, \mathbf{C}$ such that $\mathbf{AB} = \mathbf{C}$. For

$$\mathbf{r} = (1, r, \ldots, r^{k-1}), \qquad \mathbf{s} = (1, s, \ldots, s^{k-1}),$$

set

$$\mathbf{x} = \mathbf{rA}, \qquad \mathbf{y} = \mathbf{xB}, \qquad \mathbf{z} = \mathbf{rC}, \qquad \mathbf{b} = \mathbf{sB}.$$

Then $\mathbf{y} = \mathbf{z}$, and linearity of the code gives, for every $j \in I$,

$$\widehat{\mathbf{x}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{A}}[*, j]), \quad \widehat{\mathbf{y}}[j] = \mathsf{Fold}(\mathbf{x}, \widehat{\mathbf{B}}[*, j]),$$
$$\widehat{\mathbf{z}}[j] = \mathsf{Fold}(\mathbf{r}, \widehat{\mathbf{C}}[*, j]), \quad \widehat{\mathbf{b}}[j] = \mathsf{Fold}(\mathbf{s}, \widehat{\mathbf{B}}[*, j]).$$

All remaining checks are generated honestly and therefore accept.

Backend bad events. Write $\mathsf{Bad}_{\mathsf{cp}}$, $\mathsf{Bad}_{\mathsf{open}}$, $\mathsf{Bad}_{\mathsf{link}}$, and $\mathsf{Bad}_{\mathsf{code}}$ for failure of the CP-SNARK, commitment binding, CP-Link soundness, and encoding consistency, respectively, and let $\mathsf{Bad}_{\mathsf{back}}$ be their union. By Definition 6.1,

$$\Pr[\mathsf{Bad}_{\mathsf{back}}] \le \epsilon_{\mathsf{cp}}(\lambda) + \epsilon_{\mathsf{open}}(\lambda) + \epsilon_{\mathsf{link}}(\lambda) + \epsilon_{\mathsf{code}}(\lambda).$$

For the remainder of the proof, condition on $\neg\mathsf{Bad}_{\mathsf{back}}$. The input words are fixed before r, s, the intermediate words are fixed before the query tuple, and each sampled value is bound to the value opened from its commitment.

Far-input case. For an interleaved word $M \in \mathbb{F}^{k\times n}$, write

$$\Delta_{\mathsf{IRS}}(M, \mathcal{C}^k) = \min_{M' \in \mathcal{C}^k} \frac{1}{n} |\{j \in [n] : M[*, j] \ne M'[*, j]\}|.$$

For $\rho = (1, \rho, \ldots, \rho^{k-1})$, the structured-fold proximity bound is

$$\Pr_{\rho \leftarrow\$ \mathbb{F}}\big[\Delta(\rho M, \mathcal{C}) \le \tau\big] \le \frac{(k-1)n}{|\mathbb{F}|}$$

whenever $\Delta_{\mathsf{IRS}}(M, \mathcal{C}^k) > \tau$. Indeed, cancellation at a given position is governed by a nonzero polynomial in ρ of degree at most $k-1$; summing over the n positions gives the stated bound.

Suppose that an input word is τ-far from $\mathcal{C}^k$, and choose the first such word in the order $\widehat{\mathbf{A}}, \widehat{\mathbf{B}}, \widehat{\mathbf{C}}$. This choice is made from the committed words before the challenges are sampled. Use the $\mathbf{r}$-fold for $\widehat{\mathbf{A}}$ or $\widehat{\mathbf{C}}$, and the independent $\mathbf{s}$-fold for $\widehat{\mathbf{B}}$; the $\mathbf{x}\widehat{\mathbf{B}}$ fold is not needed in this case. Applying the bound to this single chosen word, the probability of a bad fold is at most

$$\frac{(k-1)n}{|\mathbb{F}|}.$$

Conditioned on the complementary event, the chosen fold is more than τ-far from the claimed intermediate codeword. Hence independent queries $I = (j_1, \ldots, j_t) \leftarrow\$ [n]^t$ all miss the disagreement set with probability at most

$$(1-\tau)^t.$$

Close-input case. Now suppose that the three words have unique decodings $\mathbf{A}^*, \mathbf{B}^*, \mathbf{C}^*$ such that

$$\Delta_{\mathsf{IRS}}(\widehat{\mathbf{A}}, \mathsf{Enc}(\mathbf{A}^*)) \le \tau,$$
$$\Delta_{\mathsf{IRS}}(\widehat{\mathbf{B}}, \mathsf{Enc}(\mathbf{B}^*)) \le \tau,$$
$$\Delta_{\mathsf{IRS}}(\widehat{\mathbf{C}}, \mathsf{Enc}(\mathbf{C}^*)) \le \tau.$$

Each error set contains at most τn interleaved columns, so

$$\mathbf{r}\widehat{\mathbf{A}} \text{ is } \tau\text{-close to } \mathsf{Enc}(\mathbf{rA}^*),$$
$$\mathbf{x}\widehat{\mathbf{B}} \text{ is } \tau\text{-close to } \mathsf{Enc}(\mathbf{xB}^*),$$
$$\mathbf{r}\widehat{\mathbf{C}} \text{ is } \tau\text{-close to } \mathsf{Enc}(\mathbf{rC}^*),$$
$$\mathbf{s}\widehat{\mathbf{B}} \text{ is } \tau\text{-close to } \mathsf{Enc}(\mathbf{sB}^*).$$

Assume $\mathbf{A}^*\mathbf{B}^* \ne \mathbf{C}^*$. Some coordinate of $\mathbf{r}(\mathbf{A}^*\mathbf{B}^* - \mathbf{C}^*)$ is a nonzero polynomial in r of degree at most $k-1$. Thus

$$\Pr_r[\mathbf{r}(\mathbf{A}^*\mathbf{B}^* - \mathbf{C}^*) = 0] \le \frac{k-1}{|\mathbb{F}|}.$$

Except with this probability, the equality $\mathbf{y} = \mathbf{z}$ and the four relations

$$\mathbf{x} = \mathbf{rA}^*, \qquad \mathbf{y} = \mathbf{xB}^*, \qquad \mathbf{z} = \mathbf{rC}^*, \qquad \mathbf{b} = \mathbf{sB}^*$$

cannot hold simultaneously. The fourth relation supplies an independent check of the middle input. The equality $\mathbf{y} = \mathbf{z}$ is checked over the complete vector and is not another sampled relation.

Choose the first false fold relation in a fixed order. This choice is made after the intermediate commitment but before I is sampled. Let c^* be the correct Reed–Solomon codeword for this relation, let $c \neq c^*$ be the claimed codeword, and let w be the folded input word. We have $\Delta(w, c^*) \leq \tau$ and $\Delta(c, c^*) \geq \delta_{\mathsf{RS}}$, whence

$$\begin{aligned}\Delta(w, c) &\geq \Delta(c, c^*) - \Delta(w, c^*)\\ &\geq \delta_{\mathsf{RS}} - \tau.\end{aligned}$$

The probability that all t queries miss the disagreement set is at most

$$(1 - \delta_{\mathsf{RS}} + \tau)^t.$$

Acceptance requires this fixed relation to pass, so no union bound over the four relations is necessary.

Combining the bounds. Combining the two cases with the backend errors yields

$$\begin{aligned}\Pr[\mathsf{Acc}] \leq\ & \epsilon_{\mathsf{cp}}(\lambda) + \epsilon_{\mathsf{open}}(\lambda) + \epsilon_{\mathsf{link}}(\lambda) + \epsilon_{\mathsf{code}}(\lambda)\\ &+ \frac{(k-1)n}{|\mathbb{F}|} + \frac{k-1}{|\mathbb{F}|}\\ &+ \max\left\{(1-\tau)^t, (1-\delta_{\mathsf{RS}}+\tau)^t\right\}.\end{aligned}$$

For the rate-one-half parameters $n = 2k$ with even k,

$$\delta_{\mathsf{RS}} = \frac{1}{2} + \frac{1}{2k}, \qquad \tau = \tau_{\mathsf{UD}} = \frac{1}{4}.$$

The larger sampling term is $(3/4)^t$, and

$$\left(\frac{3}{4}\right)^t \leq 2^{-128} \quad \iff \quad t \geq \frac{-128}{\log_2(3/4)} \approx 308.38.$$

Thus $t = 309$ gives 128-bit sampling error. This numerical choice omits the backend and field-size terms, which are assumed negligible.

Zero knowledge. Use the CP-SNARK and CP-Link simulators and replace the witness blocks by hiding Pedersen commitments. Since the Merkle layer authenticates only these commitments and reveals no openings, the resulting transcript is computationally indistinguishable from a real execution. $\square$

Appendix D. Barycentric Reed–Solomon Consistency Check

This appendix specifies the Reed–Solomon instantiation of $\mathsf{EncCheck}_{\mathcal{C}}$ used in Section 5.3. Let

$$D = \{\omega_0, \ldots, \omega_{n-1}\} \subset \mathbb{F}$$

be the public evaluation domain. A message $\mathbf{v} = (v_0, \ldots, v_{k-1}) \in \mathbb{F}^k$ defines the degree-$<k$ polynomial

$$p_{\mathbf{v}}(X) = \sum_{\ell=0}^{k-1} v_\ell X^\ell, \qquad \mathsf{Enc}(\mathbf{v}) = (p_{\mathbf{v}}(\omega_i))_{i=1}^n.$$

Given a claimed encoded word $\widehat{\mathbf{v}} \in \mathbb{F}^n$, let $P_{\widehat{\mathbf{v}}}$ be the unique degree-$<n$ polynomial satisfying

$$P_{\widehat{\mathbf{v}}}(\omega_i) = \widehat{\mathbf{v}}[i] \qquad \text{for all } i \in [n].$$

The check samples $\alpha \leftarrow\!\!\$\ \mathbb{F} \setminus D$ after the claimed word is fixed and compares $P_{\widehat{\mathbf{v}}}(\alpha)$ with $p_{\mathbf{v}}(\alpha)$.

For efficient evaluation, define the domain polynomial and barycentric weights

$$L_D(X) = \prod_{i=1}^n (X - \omega_i), \quad \lambda_i = \left(\prod_{\ell \neq i} (\omega_i - \omega_\ell)\right)^{-1}.$$

For every $\alpha \notin D$, barycentric interpolation gives

$$P_{\widehat{\mathbf{v}}}(\alpha) = L_D(\alpha) \sum_{i=1}^n \lambda_i \frac{\widehat{\mathbf{v}}[i]}{\alpha - \omega_i}.$$

Thus the concrete Reed–Solomon consistency check is

$$\begin{aligned}\mathsf{EncCheck}_{\mathsf{RS}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1 \iff & \sum_{\ell=0}^{k-1} v_\ell \alpha^\ell\\ &= L_D(\alpha) \sum_{i=1}^n \lambda_i \frac{\widehat{\mathbf{v}}[i]}{\alpha - \omega_i}.\end{aligned}$$

The right-hand side is a linear form in the claimed encoded symbols once α is fixed. Equivalently, the verifier can derive the interpolation coefficients

$$\gamma_i(\alpha) = L_D(\alpha)\lambda_i(\alpha - \omega_i)^{-1}$$

and the circuit checks

$$\sum_{\ell=0}^{k-1} v_\ell \alpha^\ell = \sum_{i=1}^n \gamma_i(\alpha)\widehat{\mathbf{v}}[i].$$

This costs $\mathcal{O}(k+n)$ field operations per checked point, after the domain-dependent weights are precomputed.

Lemma D.1 (Barycentric code-consistency soundness)**.** *If $\widehat{\mathbf{v}} \neq \mathsf{Enc}(\mathbf{v})$ and $\alpha \leftarrow\!\!\$\ \mathbb{F} \setminus D$ is sampled after $\widehat{\mathbf{v}}$ and $\mathbf{v}$ are fixed, then*

$$\Pr[\mathsf{EncCheck}_{\mathsf{RS}}(\widehat{\mathbf{v}}, \mathbf{v}; \alpha) = 1] \leq \frac{n-1}{|\mathbb{F}| - n}.$$

Proof. If $\widehat{\mathbf{v}} \neq \mathsf{Enc}(\mathbf{v})$, then the interpolation polynomial $P_{\widehat{\mathbf{v}}}$ differs from the degree-$<k$ polynomial $p_{\mathbf{v}}$. Their difference

$$Q(X) = P_{\widehat{\mathbf{v}}}(X) - p_{\mathbf{v}}(X)$$

is a nonzero polynomial of degree at most $n-1$. The check accepts exactly when $Q(\alpha) = 0$. Since α is uniform over $\mathbb{F} \setminus D$, at most $n-1$ choices of α can make the check accept, which proves the bound. $\square$

In LAMP, this check is applied independently to $\widehat{\mathbf{x}}, \widehat{\mathbf{y}}, \widehat{\mathbf{z}}$ at $\alpha_x, \alpha_y, \alpha_z$. A union bound gives the $3(n-1)/(|\mathbb{F}| - n)$ term used in Definition 6.1.

Appendix E. CP-Link Instantiation

This appendix describes the CP-Link instantiation used in our implementation. We use a pairing-based quasi-adaptive link argument, denoted QALink, for linking one SNARK-side Pedersen commitment to several external Pedersen commitments. The verifier can also batch multiple QALink verification equations into one multi-pairing check.

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a bilinear pairing over prime-order groups of order $|\mathbb{F}|$. We write the group operation in $\mathbb{G}_1$ additively, and let $g_2 \in \mathbb{G}_2$ be a generator.

Statement and witness. Fix a block count q and a block length ℓ. Let $\mathsf{ck}_S = ((G^S_{i,\nu})_{i\in[q],\nu\in[\ell]}, H^S)$ be the SNARK-side commitment key for a commitment to q concatenated blocks, and let $\mathsf{ck}_E = ((G^E_\nu)_{\nu\in[\ell]}, H^E)$ be the external Pedersen commitment key for one block.

The public statement is $\mathsf{x}_{\mathsf{link}} = ((\mathsf{ck}_S, \widetilde{\mathsf{cm}}), (\mathsf{ck}_E, \mathsf{cm}_0, \dots, \mathsf{cm}_{q-1}))$. The witness is

$$\mathsf{w}_{\mathsf{link}} = \big(\mathbf{m}_0, \dots, \mathbf{m}_{q-1}, \widetilde{o}, o_0, \dots, o_{q-1}\big),$$

where each block $\mathbf{m}_i \in \mathbb{F}^\ell$. The intended relation is that $\widetilde{\mathsf{cm}}$ commits to the ordered concatenation $\mathbf{m}_0\|\cdots\|\mathbf{m}_{q-1}$ under ck_S, while each cm_i commits to the corresponding block $\mathbf{m}_i$ under ck_E. Equivalently,

$$\begin{aligned}\widetilde{\mathsf{cm}} &= \sum_{i\in[q]}\sum_{\nu\in[\ell]} \mathbf{m}_i[\nu] G^S_{i,\nu} + \widetilde{o} H^S,\\ \mathsf{cm}_i &= \sum_{\nu\in[\ell]} \mathbf{m}_i[\nu] G^E_\nu + o_i H^E \qquad \text{for every } i \in [q].\end{aligned}$$

Setup. The setup algorithm takes $(\mathsf{ck}_S, \mathsf{ck}_E, q, \ell)$ as input. It samples $k_0, k_1, \dots, k_q, a \leftarrow\!\!\$\, \mathbb{F}^*$ and defines

$$P_{i,\nu} := k_0 G^S_{i,\nu} + k_{i+1} G^E_\nu \qquad (i \in [q],\ \nu \in [\ell]).$$

It outputs

$$\begin{aligned}\mathsf{pk}_{\mathsf{link}} &= \Big((P_{i,\nu})_{i\in[q],\,\nu\in[\ell]}, k_0 H^S, (k_{i+1} H^E)_{i\in[q]}\Big),\\ \mathsf{vk}_{\mathsf{link}} &= (C_0, C_1, \dots, C_q, A) = (ak_0 g_2, ak_1 g_2, \dots, ak_q g_2, ag_2).\end{aligned}$$

The trapdoors $k_0, \dots, k_q, a$ are discarded after setup.

Proving. Given $\mathsf{x}_{\mathsf{link}}$ and $\mathsf{w}_{\mathsf{link}}$, the prover outputs one group element

$$\pi_{\mathsf{link}} := \sum_{i\in[q]}\sum_{\nu\in[\ell]} \mathbf{m}_i[\nu] P_{i,\nu} + \widetilde{o}(k_0 H^S) + \sum_{i\in[q]} o_i(k_{i+1} H^E).$$

By construction and by the homomorphism of Pedersen commitments,

$$\pi_{\mathsf{link}} = k_0 \widetilde{\mathsf{cm}} + \sum_{i\in[q]} k_{i+1} \mathsf{cm}_i.$$

This identity is the algebraic relation checked by the verifier.

Verification. The verifier accepts if and only if

$$e(\widetilde{\mathsf{cm}}, C_0) \cdot \prod_{i\in[q]} e(\mathsf{cm}_i, C_{i+1}) = e(\pi_{\mathsf{link}}, A).$$

Equivalently, the verifier checks

$$e(\widetilde{\mathsf{cm}}, C_0) \cdot \prod_{i\in[q]} e(\mathsf{cm}_i, C_{i+1}) \cdot e(\pi_{\mathsf{link}}, -A) = 1_{\mathbb{G}_T}.$$

Correctness. For an honestly generated proof, we have $\pi_{\mathsf{link}} = k_0\widetilde{\mathsf{cm}} + \sum_{i\in[q]} k_{i+1}\mathsf{cm}_i$. Therefore, by bilinearity,

$$\begin{aligned}e(\pi_{\mathsf{link}}, A) &= e\left(k_0\widetilde{\mathsf{cm}} + \sum_{i\in[q]} k_{i+1}\mathsf{cm}_i, ag_2\right)\\ &= e(\widetilde{\mathsf{cm}}, ak_0 g_2) \cdot \prod_{i\in[q]} e(\mathsf{cm}_i, ak_{i+1} g_2)\\ &= e(\widetilde{\mathsf{cm}}, C_0) \cdot \prod_{i\in[q]} e(\mathsf{cm}_i, C_{i+1}).\end{aligned}$$

Thus the verifier accepts.

Soundness. QALink is a quasi-adaptive linear proof for the subspace relation induced by the commitment keys $(\mathsf{ck}_S, \mathsf{ck}_E)$, block count q, and block length ℓ. Under the standard quasi-adaptive linear-algebra assumption for the above pairing CRS, any accepting proof for a statement outside this relation implies a nontrivial linear relation among the hidden setup trapdoors $k_0, \dots, k_q, a$. Thus the argument is sound except with negligible probability in the group order.

Verifier cost. For one link statement with q external commitments, verification consists of $q + 2$ pairing terms: one term for $\widetilde{\mathsf{cm}}$, q terms for $\mathsf{cm}_0, \dots, \mathsf{cm}_{q-1}$, and one term for π_{link} against $-A$. A multi-pairing implementation evaluates these terms with $q + 2$ Miller-loop inputs and one final exponentiation. The proof contains one $\mathbb{G}_1$ element, and the verifying key contains $q + 2$ $\mathbb{G}_2$ elements.

Batching pairing equations. Suppose the verifier must check s independent QALink statements. For the h-th statement, let

$$E_h := e(\widetilde{\mathsf{cm}}^{(h)}, C_0^{(h)}) \cdot \prod_{i\in[q_h]} e(\mathsf{cm}_i^{(h)}, C_{i+1}^{(h)}) \cdot e(\pi_{\mathsf{link}}^{(h)}, -A^{(h)}).$$

The unbatched verifier checks $E_h = 1_{\mathbb{G}_T}$ for every h. The batched verifier first derives nonzero coefficients $\lambda_1, \dots, \lambda_s \in \mathbb{F}^*$ by hashing all public statements, verifying keys, proofs, and the surrounding transcript context. It then checks the single equation

$$\prod_{h=1}^{s} E_h^{\lambda_h} = 1_{\mathbb{G}_T}.$$

Operationally, this is implemented by scaling the $\mathbb{G}_1$ input of every pairing term in the h-th equation by λ_h, and then running one multi-pairing check over all scaled terms:

$$\prod_{h=1}^{s} e(\lambda_h \widetilde{\mathsf{cm}}^{(h)}, C_0^{(h)}) \cdot \prod_{h=1}^{s} \prod_{i\in[q_h]} e(\lambda_h \mathsf{cm}_i^{(h)}, C_{i+1}^{(h)}) \cdot \prod_{h=1}^{s} e(\lambda_h \pi_{\mathsf{link}}^{(h)}, -A^{(h)}) = 1_{\mathbb{G}_T}.$$

The batched verifier uses $\sum_{h=1}^{s}(q_h + 2)$ Miller-loop terms and one final exponentiation. It also performs $\sum_{h=1}^{s}(q_h + 2)$ scalar multiplications in $\mathbb{G}_1$ to apply the batching coefficients. If all individual equations are valid, then the batched equation is valid by bilinearity. If at least one individual equation is invalid, write $E_h = g_T^{\delta_h}$ in the prime-order target group. The batched equation accepts only if $\sum_h \lambda_h \delta_h = 0$. Since at least one $\delta_h \neq 0$, a fresh random nonzero coefficient vector satisfies this linear equation with probability at most $1/(|\mathbb{F}|-1)$. In the non-interactive implementation this probability is interpreted in the random oracle model for the Fiat–Shamir-derived batching coefficients.

Appendix F. LAMP Results Across Code Rates

Table 7 reports the performance of LAMP for code rates $\rho \in \{1/2, 1/4, 1/8\}$. The setup time is the sum of the Groth16 and CP-Link setup times. Encoding and commitment generation are reported separately, where commitment is the sum of the ABC and XYZ commitment times and excludes encoding. Proving time is the sum of the Merkle, Groth16, and CP-Link proof-generation times and excludes both encoding and commitment generation. For $k = 2^7$ and $\rho = 1/2$, the codeword length is $n = 256$, which is smaller than the required number of queries $t = 309$; therefore, no measurement is reported for this parameter set.

k	ρ	n	t	**Constraints**	**Setup (s)**	**Encode (s)**	**Commit (s)**	**Prove (s)**	**Verify (s)**	**Proof (B)**
2^7	$1/2$	256	309	–	–	–	–	–	–	–
	$1/4$	512	189	107,502	6.71	0.02	0.15	0.98	0.08	36,484
	$1/8$	1,024	155	95,916	5.97	0.03	0.29	0.86	0.07	40,964
2^8	$1/2$	512	309	329,377	21.31	0.04	0.23	2.67	0.13	50,308
	$1/4$	1,024	189	211,456	13.38	0.06	0.47	1.73	0.08	48,580
	$1/8$	2,048	155	188,615	11.85	0.10	0.89	1.61	0.07	49,220
2^9	$1/2$	1,024	309	655,245	42.48	0.14	0.78	4.80	0.14	65,860
	$1/4$	2,048	189	420,117	26.60	0.21	1.53	3.23	0.08	57,412
	$1/8$	4,096	155	374,648	23.06	0.31	3.04	2.82	0.07	59,652
2^{10}	$1/2$	2,048	309	1,306,972	85.59	0.46	2.80	9.30	0.13	80,772
	$1/4$	4,096	189	837,448	52.38	0.69	5.46	5.95	0.08	69,060
	$1/8$	8,192	155	746,714	46.42	1.08	10.76	5.21	0.07	68,356
2^{11}	$1/2$	4,096	309	2,609,193	168.33	1.51	8.38	17.59	0.13	99,012
	$1/4$	8,192	189	1,671,348	105.08	2.23	16.25	11.07	0.08	80,580
	$1/8$	16,384	155	1,490,211	91.52	3.88	32.01	9.49	0.07	77,636
2^{12}	$1/2$	8,192	309	5,214,877	335.68	5.13	29.85	34.96	0.13	119,556
	$1/4$	16,384	189	3,339,901	208.62	8.14	57.61	21.67	0.08	91,332
	$1/8$	32,768	155	2,977,840	182.35	14.29	111.57	18.38	0.07	86,148
2^{13}	$1/2$	16,384	309	10,426,236	677.08	18.09	107.37	68.44	0.13	136,260
	$1/4$	32,768	189	6,677,016	425.24	31.59	209.37	42.59	0.08	103,108
	$1/8$	65,536	155	5,953,098	369.93	59.08	406.62	36.68	0.07	97,348

TABLE 7. PERFORMANCE OF LAMP ACROSS CODE RATES.